INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO

BRIAN GUSTAVO DE OLIVEIRA BRANDÃO

MEGAGAME: SISTEMA DE *MARKETPLACE* DE JOGOS

CAMPOS DO JORDÃO
2026

BRIAN GUSTAVO DE OLIVEIRA BRANDÃO

MEGAGAME: SISTEMA DE *MARKETPLACE* DE JOGOS

Projeto Integrador apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), Campus Campos do Jordão, como requisito parcial para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Orientador: Prof. Dr. Helton Hugo de Carvalho Júnior

Campos do Jordão
2026

BRIAN GUSTAVO DE OLIVEIRA BRANDÃO

MEGAGAME: SISTEMA DE *MARKETPLACE* DE JOGOS

Projeto Integrador apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), Campus Campos do Jordão, como requisito parcial para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Aprovado em: ___/___/_____.

BANCA EXAMINADORA

Prof. Dr. Helton Hugo de Carvalho Júnior
Orientador
Instituto Federal de São Paulo (IFSP)

Avaliador(a)

Avaliador(a)

RESUMO

Este trabalho apresenta o desenvolvimento do MegaGame, um *marketplace customer-to-customer* (C2C) voltado ao comércio de jogos, principalmente os eletrônicos, e de itens associados. O projeto parte da constatação de que esse mercado cresce de forma expressiva, tanto em retorno financeiro quanto em número de interessados, e de que, no Brasil, a venda desses produtos está espalhada por diversas plataformas generalistas. Com esse cenário em mente, surgiu a ideia de se criar um sistema com enfoque nesses produtos, assim fornecendo um local de comércio centralizado para entusiastas. Para isso, inicialmente foram levantados os requisitos funcionais e não funcionais, modelados os casos de uso com a *Unified Modeling Language* (UML) e projetado o banco de dados relacional. Aí, a implementação utilizou a linguagem Python com adição do *framework* Django, empregando a técnica de *server-side rendering*, além do banco de dados SQLite e HTML/CSS/JavaScript na interface, com pagamentos sendo processados pela API do Mercado Pago. O resultado foi uma plataforma que permite compras diretas, leilões e trocas, contemplando o fluxo completo de um pedido (incluindo devolução e mediação de disputas) e gestão financeira/administrativa via *dashboards*, também incluindo funcionalidades como avaliação de usuários e produtos, gamificação e afins. O sistema funciona de ponta a ponta em ambiente de testes, mas sua entrada em produção foi inviabilizada por uma questão de hospedagem e de integração com serviços reais de rastreio e geolocalização.

Palavras-chave: *marketplace*; jogos eletrônicos; comércio eletrônico; Django; engenharia de *software*.

ABSTRACT

This work presents the development of MegaGame, a customer-to-customer (C2C) marketplace focused on the trade of games, especially electronic games, and related items. The project stems from the observation that this market is growing significantly, both in financial return and in number of consumers, and that, in Brazil, the sale of these products is scattered across several general-purpose platforms. With that situation in mind, the idea of creating a system with a focus in such products was born, in order to provide enthusiasts with a centralized commerce spot. To that end, initially, the functional and non-functional requirements were gathered, the use cases were modeled with the Unified Modeling Language (UML) and the relational database was designed. Then, the implementation used the Python language with the addition of the Django framework, employing server-side rendering, plus the SQLite database and HTML/CSS/JavaScript on the interface, with payments being processed by Mercado Pago's API. The result is a platform which allows direct purchases, auctions and trades, encompassing the entire ordering flow (including returns and disputes) and financial/administrative managing via dashboards, also including functionalities such as seller and product reviews, gamification and so forth. The system works end to end in a test environment, but putting it into production turned out to be unviable due to problems with hosting and integrating real services for parcel tracking and geolocalization.

Keywords: marketplace; video games; e-commerce; Django; software engineering.

LISTA DE FIGURAS

Figura 1 – Logo do Supabase 14
Figura 2 – Logo da UML 18
Figura 3 – Logo do Django 19
Figura 4 – Logo do Python 19
Figura 5 – Painel administrativo do Django no MegaGame 20
Figura 6 – Logo do SQLite 21
Figura 7 – Diagrama de casos de uso do gestor 30
Figura 8 – Diagrama de casos de uso do comprador: compras e pós-venda . . 31
Figura 9 – Diagrama de casos de uso do comprador: leilões, trocas e recompensas 32
Figura 10 – Diagrama de casos de uso do vendedor: conta, anúncios e leilões . 33
Figura 11 – Diagrama de casos de uso do vendedor: vendas e pós-venda . . . 34
Figura 12 – Estrutura de arquivos na raiz do projeto 49
Figura 13 – DER: usuários e catálogo 51
Figura 14 – DER: pedidos e pós-venda 52
Figura 15 – DER: leilões e trocas 53
Figura 16 – DER: cupons, MegaCoins e gestão 53
Figura 17 – *Wireframe* da página inicial do MegaGame 74
Figura 18 – *Wireframe* da página de categorias do MegaGame 74
Figura 19 – *Wireframe* da página de anúncios do MegaGame 75
Figura 20 – *Wireframe* da página de cadastro do MegaGame 75
Figura 21 – *Wireframe* do menu do usuário do MegaGame 76
Figura 22 – *Wireframe* do manual de usuário do MegaGame 76
Figura 23 – Captura de tela da página inicial do MegaGame 78
Figura 24 – Captura de tela da busca com filtros do MegaGame 79
Figura 25 – Captura de tela da página de produto do MegaGame 80
Figura 26 – Captura de tela de um leilão no MegaGame 81
Figura 27 – Captura de tela de uma negociação de troca no MegaGame 82
Figura 28 – Captura de tela da finalização do pedido no MegaGame 83
Figura 29 – Captura de tela da carteira de MegaCoins 84
Figura 30 – Captura de tela do *dashboard* do gestor do MegaGame 85
Figura 31 – Captura de tela do menu do manual de usuário 87
Figura 32 – Captura de tela de um tutorial do manual de usuário 88

LISTA DE QUADROS

Quadro 1 – Requisitos funcionais 24
Quadro 2 – Requisitos não funcionais 26
Quadro 3 – Casos de uso do MegaGame 27
Quadro 4 – Caso de uso: Cadastrar conta 34
Quadro 5 – Caso de uso: Efetuar login 35
Quadro 6 – Caso de uso: Navegar pelas categorias 35
Quadro 7 – Caso de uso: Pesquisar produto 35
Quadro 8 – Caso de uso: Editar perfil 35
Quadro 9 – Caso de uso: Analisar o *dashboard* administrativo 36
Quadro 10 – Caso de uso: Alterar parâmetros da plataforma 36
Quadro 11 – Caso de uso: Adicionar produto ao carrinho 37
Quadro 12 – Caso de uso: Fazer pedido 37
Quadro 13 – Caso de uso: Receber produto 37
Quadro 14 – Caso de uso: Avaliar vendedor 38
Quadro 15 – Caso de uso: Avaliar produto 38
Quadro 16 – Caso de uso: Cancelar pedido 38
Quadro 17 – Caso de uso: Analisar *dashboard* de vendas 38
Quadro 18 – Caso de uso: Ver pedidos recebidos 39
Quadro 19 – Caso de uso: Preparar entrega 39
Quadro 20 – Caso de uso: Publicar anúncio 39
Quadro 21 – Caso de uso: Editar anúncio 40
Quadro 22 – Caso de uso: Retirar anúncio do ar 40
Quadro 23 – Caso de uso: Deletar anúncio 40
Quadro 24 – Caso de uso: Conectar conta do Mercado Pago 41
Quadro 25 – Caso de uso: Rastrear pedido 41
Quadro 26 – Caso de uso: Solicitar devolução 42
Quadro 27 – Caso de uso: Responder solicitação de devolução 42
Quadro 28 – Caso de uso: Abrir disputa 42
Quadro 29 – Caso de uso: Resolver disputa 43
Quadro 30 – Caso de uso: Criar leilão 43
Quadro 31 – Caso de uso: Gerenciar leilão 44
Quadro 32 – Caso de uso: Dar lance 44
Quadro 33 – Caso de uso: Comprar agora 44
Quadro 34 – Caso de uso: Acompanhar leilão 45
Quadro 35 – Caso de uso: Propor troca 45
Quadro 36 – Caso de uso: Responder proposta de troca 46
Quadro 37 – Caso de uso: Gerenciar cupons 46

Quadro 38 – Caso de uso: Salvar cupom 46
Quadro 39 – Caso de uso: Consultar carteira de MegaCoins 47
Quadro 40 – Caso de uso: Fazer *check-in* diário 47
Quadro 41 – Caso de uso: Registrar despesa 47
Quadro 42 – Caso de uso: Exportar relatório financeiro 48
Quadro 43 – Dicionário de dados: Usuários (auth_user) 54
Quadro 44 – Dicionário de dados: Perfis de usuário (accounts_profile) 55
Quadro 45 – Dicionário de dados: Contas do Mercado Pago (accounts_mercadopagoaccount) 55
Quadro 46 – Dicionário de dados: Avaliações de vendedores (accounts_sellerreview) 55
Quadro 47 – Dicionário de dados: Categorias (catalog_category) 56
Quadro 48 – Dicionário de dados: Produtos (catalog_product) 56
Quadro 49 – Dicionário de dados: Variações de produtos (catalog_productvariant) 57
Quadro 50 – Dicionário de dados: Imagens de produtos (catalog_productimage) 58
Quadro 51 – Dicionário de dados: Avaliações de produtos (catalog_productreview) 58
Quadro 52 – Dicionário de dados: Carrinhos (orders_cart) 58
Quadro 53 – Dicionário de dados: Itens do carrinho (orders_cartitem) 59
Quadro 54 – Dicionário de dados: Pedidos (orders_order) 59
Quadro 55 – Dicionário de dados: Comissões (orders_commission) 61
Quadro 56 – Dicionário de dados: Solicitações de devolução (orders_returnrequest) 61
Quadro 57 – Dicionário de dados: Imagens de solicitações de devolução (orders_returnrequestimage) 62
Quadro 58 – Dicionário de dados: Disputas (orders_dispute) 62
Quadro 59 – Dicionário de dados: Mensagens das disputas (orders_disputemessage) 63
Quadro 60 – Dicionário de dados: Evidências de disputa (orders_disputeevidence) 63
Quadro 61 – Dicionário de dados: Leilões (auctions_auction) 63
Quadro 62 – Dicionário de dados: Lances (auctions_bid) 65
Quadro 63 – Dicionário de dados: Leilões acompanhados (auctions_auctionwatch) 65
Quadro 64 – Dicionário de dados: Ofertas de segunda chance (auctions_secondchanceoffer) 66
Quadro 65 – Dicionário de dados: Propostas de troca (trades_tradeproposal) . . 66
Quadro 66 – Dicionário de dados: Itens oferecidos em trocas (trades_tradeitem) 67
Quadro 67 – Dicionário de dados: Eventos de troca (trades_tradeevent) 68
Quadro 68 – Dicionário de dados: Cupons (coupons_coupon) 69
Quadro 69 – Dicionário de dados: Usos de cupons (coupons_couponredemption) 70
Quadro 70 – Dicionário de dados: Cupons salvos (coupons_savedcoupon) . . . 70

Quadro 71 – Dicionário de dados: Movimentações de MegaCoins (rewards_cointransaction) 70
Quadro 72 – Dicionário de dados: Despesas (orders_expense) 71
Quadro 73 – Dicionário de dados: Configurações da plataforma (orders_platformconfig) 72

LISTA DE ABREVIATURAS E SIGLAS

API	*Application Programming Interface*
C2C	*Customer-to-customer*
CSS	*Cascading Style Sheets*
CSU	Caso de uso
DER	Diagrama entidade-relacionamento
FK	*Foreign key* (chave estrangeira)
GMV	*Gross merchandise value* (valor bruto das mercadorias vendidas)
HTML	*HyperText Markup Language*
ID	Identificador
IDE	*Integrated development environment* (ambiente de desenvolvimento integrado)
IEEE	Instituto de Engenheiros Eletricistas e Eletrônicos
IFSP	Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
ISO	*International Organization for Standardization*
LGPD	Lei Geral de Proteção de Dados Pessoais
OMG	*Object Management Group*
PDF	*Portable Document Format*
PK	*Primary key* (chave primária)
RF	Requisito funcional
RNF	Requisito não funcional
SGBD	Sistema de gerenciamento de banco de dados
SQL	*Structured Query Language*
SSR	*Server-side rendering* (renderização no lado do servidor)
SUAP	Sistema Unificado de Administração Pública
UML	*Unified Modeling Language*
URL	*Uniform Resource Locator*

SUMÁRIO

1 INTRODUÇÃO 12
1.1 Objetivos 12
1.1.1 OBJETIVO GERAL 12
1.1.2 OBJETIVOS ESPECÍFICOS 12
1.2 Justificativa 13
1.3 Delimitação do escopo 13
1.4 Aspectos metodológicos 14
1.5 Aporte teórico 15

2 FUNDAMENTAÇÃO TEÓRICA 16
2.1 Engenharia de software 16
2.1.1 ANÁLISE DE REQUISITOS 16
2.1.2 MODELAGEM 17
2.2 Linguagens e ferramentas utilizadas 17
2.2.1 UML 18
2.2.2 DJANGO 18
2.2.3 SQLITE 20
2.3 Desenvolvimento do projeto 21

3 PROJETO PROPOSTO 22
3.1 Considerações iniciais 22
3.2 Perfis de usuário 22
3.2.1 GESTOR 22
3.2.2 COMPRADOR 23
3.2.3 VENDEDOR 23
3.3 Requisitos 23
3.4 Casos de uso 27
3.4.1 DIAGRAMAS DE CASOS DE USO 29
3.4.2 DESCRIÇÕES DE CASOS DE USO 34
3.5 Arquitetura 48
3.6 Projeto de dados 50
3.6.1 DICIONÁRIO DE DADOS 54
3.7 Protótipos de baixo nível 73
3.8 Interfaces 76
3.9 Implementação 85
3.10 Testes 86
3.11 Manual de usuário 86

4 CONCLUSÃO 89

REFERÊNCIAS 90

GLOSSÁRIO 92

1 INTRODUÇÃO

De acordo com a Newzoo (2025), a indústria de jogos eletrônicos forma atualmente um dos maiores mercados da área de entretenimento, movimentando centenas de bilhões de dólares por ano, e o Brasil figura entre os países que mais contribuem para esse resultado, sustentado por uma forte comunidade *gamer*.

Sendo assim, este projeto tem como objetivo apresentar o MegaGame, um *marketplace* do tipo *customer-to-customer* (C2C), isto é, de comércio entre consumidores, voltado para essa audiência. Além da oportunidade comercial, o desenvolvimento do site também foi motivado pela observação do autor de que seria útil existir um local centralizado para o comércio de itens relacionados ao tema, tendo em vista a grande dispersão através de diversas plataformas (OLX, Mercado Livre, Shopee, dentre outras) em decorrência da falta de um serviço dedicado ao segmento.

Para o desenvolvimento do MegaGame, foram utilizados a linguagem Python (com o *framework* Django) e o sistema de gerenciamento de banco de dados (SGBD) SQLite, com suporte ao PostgreSQL por meio da plataforma Supabase, conforme detalhado na Seção 1.3.

1.1 Objetivos

Nesta seção, são apresentados o objetivo geral e os objetivos específicos deste trabalho.

1.1.1 OBJETIVO GERAL

O objetivo deste trabalho é desenvolver o MegaGame, uma plataforma em que compradores e vendedores negociem jogos e itens relacionados diretamente entre si, e documentar seu processo de desenvolvimento, seus requisitos, sua modelagem e suas funcionalidades.

1.1.2 OBJETIVOS ESPECÍFICOS

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

a) levantar os requisitos funcionais e não funcionais da plataforma e modelar seus casos de uso com a UML;

b) projetar o banco de dados relacional e documentar suas tabelas em dicionários de dados;

c) implementar o cadastro e a autenticação de usuários, permitindo que uma mesma conta atue como compradora e vendedora;

d) permitir a publicação e a gestão de anúncios, com categorias, variações de produto e rascunhos;

e) implementar o fluxo de compra, do carrinho ao acompanhamento do pedido, com pagamento pelo Mercado Pago e opção de retirada em mãos;

f) oferecer formas de negociação além da venda direta, por meio de leilões e de propostas de troca entre usuários;

g) tratar o pós-venda com cancelamentos, devoluções e disputas mediadas pelo gestor;

h) estimular o uso da plataforma com cupons de desconto, a moeda virtual MegaCoins e avaliações de produtos e vendedores;

i) disponibilizar *dashboards* financeiros para vendedores e gestores, incluindo o registro de despesas da plataforma e a exportação de relatórios em PDF.

1.2 Justificativa

Conforme citado anteriormente, a justificativa deste trabalho se dá pela observação do autor de que há uma lacuna (ao menos no cenário brasileiro) na indústria de *games* que pode ser preenchida por uma plataforma com as características do MegaGame, além da crescente viabilidade comercial do setor.

A plataforma atende a cada uma das partes envolvidas: o comprador encontra num só lugar os produtos do segmento, o vendedor conta com mais de uma forma de negociar, seja por venda direta, leilão ou troca, e a plataforma obtém receita com a comissão cobrada sobre as vendas.

1.3 Delimitação do escopo

Nesta seção, são delimitados o alcance do MegaGame e as limitações assumidas no desenvolvimento deste trabalho.

O MegaGame foi planejado para usar o PostgreSQL hospedado na plataforma Supabase. Durante o desenvolvimento, porém, o *firewall* da rede do IFSP passou a bloquear o acesso ao serviço, o que impedia o uso do sistema nas dependências do câmpus. Por isso, o banco de dados principal passou a ser o SQLite, armazenado em um arquivo local. A conexão com o Supabase continua prevista nas configurações do projeto e pode ser reativada sem alterações nos modelos de dados. O Supabase,

criado em 2020 por Paul Copplestone e Ant Wilson como alternativa de código aberto ao Firebase, do Google, tem seu logotipo apresentado na Figura 1.

Figura 1 – Logo do Supabase



Fonte: Supabase ([2026?]).

O rastreio das entregas também é simulado. Para consultar os eventos dos Correios, foram avaliados três serviços: o Linketrack, que se tornou obsoleto; o SiteRastreio, que passou a cobrar por requisição após sua integração com a Wonca; e o Melhor Rastreio, que não oferece API e só acompanha pacotes postados com etiquetas do Melhor Envio. A integração com o SiteRastreio chegou a ser implementada, mas foi desativada por causa do custo. Em seu lugar, o sistema monta uma linha do tempo fictícia a partir do código de rastreio, das cidades do vendedor e do comprador e do tempo decorrido desde o envio, reproduzindo as etapas exibidas pelos Correios. Também não há cálculo de frete: o valor do pedido considera apenas o preço do produto e os descontos aplicados.

Por fim, o sistema ainda não está em produção. Ele é executado apenas em ambiente local, sem hospedagem na *web*, e a integração com o Mercado Pago opera no modo *sandbox*, o ambiente de testes da plataforma, em que os pagamentos usam contas e cartões fictícios e não movimentam dinheiro. Para que as notificações de pagamento enviadas pelo Mercado Pago (*webhooks*) cheguem ao servidor local, ele é exposto temporariamente à internet por um túnel do ngrok.

1.4 Aspectos metodológicos

As ferramentas utilizadas durante a criação do projeto foram a linguagem de programação Python no *back end*, através do *framework* Django e utilizando a prática de renderização no lado do servidor (*server-side rendering* – SSR); a linguagem de programação JavaScript no *front end*, aliada à linguagem de marcação *HyperText Markup Language* (HTML) e à linguagem de estilo *Cascading Style Sheets* (CSS); e, além disso, o SGBD SQLite, com a opção de usar o PostgreSQL hospedado na nuvem pela plataforma *online* Supabase.

Já os diagramas da *Unified Modeling Language* (UML), linguagem de modelagem visual, exibidos no documento foram desenhados com o TikZ, um pacote do LaTeX para a criação de gráficos (Tantau, 2026).

1.5 Aporte teórico

Para os estudos teóricos acerca do tema abordado, foram consultados alguns dos livros mais populares e acessíveis sobre tópicos como a UML, a engenharia de *software* e o gerenciamento de bancos de dados, além de fontes virtuais que comprovem alguns dos conceitos que embasam o projeto como um todo.

Mais especificamente, os escritos de autores como Coronel, Pressman e Sommerville foram os responsáveis por ajudar a definir os princípios guiadores do trabalho, graças aos métodos, práticas e paradigmas estabelecidos e discutidos por tais obras.

2 FUNDAMENTAÇÃO TEÓRICA

Nesta seção, serão apresentadas em maiores detalhes as tecnologias e teorias empregadas durante a construção do projeto.

2.1 Engenharia de software

O conceito de engenharia de *software* é utilizado para se referir a uma área da computação voltada para a aplicação de certos princípios e ferramentas, com o intuito de esquematizar, desenvolver e gerenciar sistemas. Seu propósito é garantir que todos os requisitos determinados no início de um projeto sejam cumpridos de forma eficaz, mas também de forma eficiente quando possível; isto é, assegurando que tudo esteja dentro do prazo estabelecido, respeitando os limites orçamentários e se encaixando em padrões de qualidade do mercado (como as normas da *International Organization for Standardization* – ISO, por exemplo).

Atualmente, no contexto acadêmico, são duas as principais referências neste setor – as obras de Pressman e Maxim, junto das obras de Sommerville. Pressman e Maxim (2021, p. 59) dizem, acerca do tema:

> O Instituto de Engenheiros Eletricistas e Eletrônicos (IEEE) elaborou a seguinte definição para engenharia de *software*: [...] a aplicação de uma abordagem sistemática, disciplinada e quantificável no desenvolvimento, na operação e na manutenção de *software*; isto é, a aplicação de engenharia ao *software*.

Já Sommerville (2011, p. 22) afirma:

> A engenharia de *software* tem por objetivo apoiar o desenvolvimento profissional de *software*, mais do que a programação individual. Ela inclui técnicas que apoiam especificação, projeto e evolução de programas, que normalmente não são relevantes para o desenvolvimento de *software* pessoal.

As etapas da engenharia de *software* são organizadas em processos, para os quais são levantados os requisitos correspondentes. Assim, os melhores métodos possíveis podem ser empregados para cumpri-los.

2.1.1 ANÁLISE DE REQUISITOS

Existem três tipos diferentes de requisito, sendo dois deles os principais (mais precisamente, os funcionais e os não funcionais). Primeiramente, os requisitos funcionais descrevem as funcionalidades exatas que um sistema deve possuir. Por exemplo,

um site que trabalhe com perfis de usuário deve permitir que seus clientes cadastrem contas para si e depois efetuem o *login* nelas.

Já os requisitos não funcionais referem-se aos atributos de qualidade que o sistema deve alcançar. Por exemplo, suportar X usuários simultâneos, carregar todo o conteúdo em até Y segundos, garantir conformidade com leis de proteção de dados de determinado território, dentre outros casos.

Por fim, existem também os requisitos de domínio. Tecnicamente, eles se enquadram em uma das categorias anteriores, mas vale a pena ressaltá-los à parte, pois correspondem às regras de negócio de um determinado setor; isto é, são formados a partir da análise daquela área em específico, para garantir que suas particularidades estejam sendo observadas e tratadas.

Para obter todos os requisitos, é realizada uma ação conhecida como levantamento, que pode englobar várias táticas. Análise de mercado, entrevistas com peritos, discussões de ideias com o cliente e/ou outros desenvolvedores (processo também conhecido como *brainstorming*) e demais práticas são passíveis de serem utilizadas durante essa etapa do planejamento.

2.1.2 MODELAGEM

A partir do momento em que os requisitos foram todos levantados e já existe uma ideia do rumo que o projeto deve tomar, é uma boa prática fazer a modelagem do *software*, através de ferramentas como a UML (cujo uso será mais detalhado posteriormente, por se tratar de uma linguagem utilizada durante a construção deste trabalho em específico).

Modelagem, neste cenário, significa representar o sistema sendo desenvolvido através de recursos visuais, facilitando sua compreensão, análise e posterior implementação. É especialmente útil para trabalhos em equipe ou que envolvem *stakeholders* externos, mas também pode se mostrar de suma importância mesmo em criações individuais.

Ela pode abranger diversas áreas. No ecossistema da UML, por exemplo, existem diagramas de casos de uso, diagramas de classe (que representam a estrutura do sistema, através do conceito da programação orientada a objetos), diagramas de sequência (para detalhar as atividades que o sistema realiza), dentre outros.

2.2 Linguagens e ferramentas utilizadas

Nesta seção, são descritas as linguagens, os *frameworks* e as plataformas empregados na modelagem e na implementação do MegaGame.

2.2.1 UML

UML é a sigla de *Unified Modeling Language* (em português, “linguagem unificada de modelagem”), nome de uma linguagem de modelagem desenvolvida na década de 1990 por Grady Booch, James Rumbaugh e Ivar Jacobson, a partir da unificação de suas três linguagens desenvolvidas de forma independente em um “pacote” coeso. Seu logotipo é apresentado na Figura 2.

Figura 2 – Logo da UML



Fonte: Object Management Group ([2026?]).

Desde 1997, a UML é gerida e padronizada pelo *Object Management Group* (OMG). Possui diversos usuários no meio de desenvolvimento de *software*, inclusive Gilleanes Guedes, cujas obras servem como um guia para sua utilização ideal. Guedes (2018, p. 24) afirma:

> Um modelo de software captura uma visão de um sistema físico; é uma abstração do sistema com um certo propósito, como descrever aspectos estruturais ou comportamentais do *software*. Esse propósito determina o que deve ser incluído no modelo e o que é considerado irrelevante. Assim, um modelo descreve completamente aqueles aspectos do sistema físico relevantes ao propósito do modelo, no nível apropriado de detalhe.

Os diagramas da UML são divididos em dois tipos: os estruturais e os comportamentais. Os primeiros focam na organização estática do sistema (como os diagramas de classes, objetos, componentes, implantação etc.) e os últimos focam na dinâmica e interações do sistema (como os diagramas de casos de uso, sequência, estados etc.).

2.2.2 DJANGO

O Django é um *framework* para a linguagem Python, desenvolvido em 2005 por Adrian Holovaty e Simon Willison, sendo mantido desde então pela Django Software Foundation. O Python, por sua vez, foi desenvolvido em 1991 pelo neerlandês Guido van Rossum para suceder uma linguagem anterior conhecida como ABC, e é uma das

linguagens de programação mais populares e acessíveis do mundo. Os logotipos de ambos são exibidos nas Figuras 3 e 4.

Figura 3 – Logo do Django



Fonte: Django Software Foundation ([2026?b]).

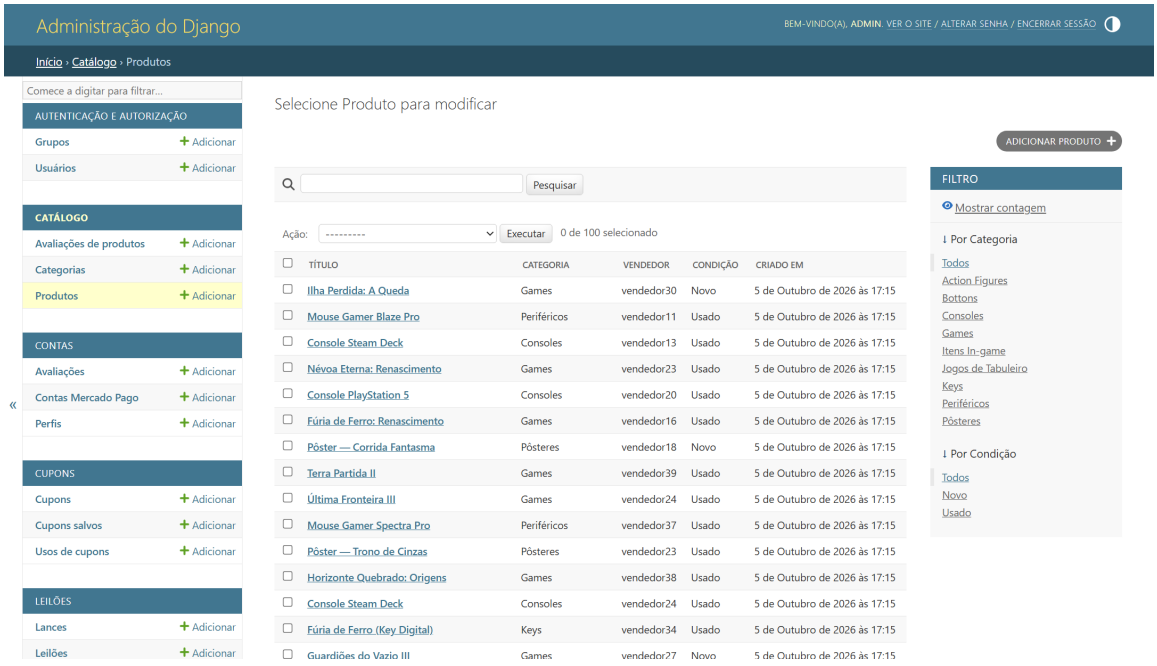
Figura 4 – Logo do Python



Fonte: Python Software Foundation ([2026?]).

Em comparação com outros *frameworks web* para Python, como o Flask, o Django se destaca por já trazer prontos recursos que em outros dependem de extensões, como o mapeamento dos modelos para o banco de dados, a autenticação de usuários e o painel administrativo. É utilizado por diversas empresas, incluindo o próprio Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), em sua plataforma virtual Sistema Unificado de Administração Pública (SUAP). Trabalha com o conceito de componentes (similar a alguns outros *frameworks web* populares, como Angular, este em específico sendo visto no decorrer do curso), permitindo alta modularização, e fornece uma interface administrativa embutida para facilitar a gestão, conforme mostra a Figura 5.

Figura 5 – Painel administrativo do Django no MegaGame



Fonte: elaborado pelo próprio autor.

Através da prática do SSR, é possível utilizá-lo para exibir o site sem necessidade de *frameworks* adicionais de *front end*. Sendo assim, tratando-se da questão *web*, a combinação usual de HTML/CSS/JavaScript já foi suficiente para atingir todas as metas almejadas.

2.2.3 SQLITE

O SQLite é uma biblioteca escrita na linguagem C que implementa um SGBD relacional baseado na *Structured Query Language* (SQL), sem a necessidade de um servidor separado. O banco de dados inteiro, com suas tabelas, índices e demais estruturas, fica guardado em um único arquivo, lido e escrito diretamente pela aplicação (SQLite, [2026?a]).

O projeto foi iniciado em maio de 2000 e tem Richard Hipp como principal desenvolvedor (SQLite, [2026?a]; SQLite, [2026?d]). Seu código está em domínio público, o que permite o uso livre para qualquer finalidade, comercial ou privada. Segundo os próprios desenvolvedores, o SQLite é provavelmente mais usado do que todos os outros mecanismos de banco de dados somados, estando presente, por exemplo, em todos os aparelhos Android e iPhone e nos navegadores Firefox, Chrome e Safari (SQLite, [2026?c]). Seu logotipo é apresentado na Figura 6.

Figura 6 – Logo do SQLite



Fonte: SQLite ([2026?a]).

No Django, o SQLite é o banco configurado por padrão, e o Python já inclui o suporte a ele, sem necessidade de instalação adicional (Django Software Foundation, [2026?d]). Como o Django cria as tabelas a partir dos modelos do projeto por meio do processo de *migration*, a troca entre o SQLite e o PostgreSQL descrita na Seção 1.3 depende apenas da configuração do banco, e as mesmas tabelas são geradas em qualquer um dos dois.

2.3 Desenvolvimento do projeto

O projeto começou com a ideia inicial, que surgiu a partir dos gostos pessoais do autor deste trabalho, com o auxílio das sugestões do orientador da disciplina. A partir dela, foi feita uma pesquisa de mercado para determinar as regras de negócio do setor, além da confecção de *wireframes* para as principais páginas do site, de modo a definir o *design* geral, conforme apresentado na Seção 3.7.

Em seguida, partiu-se para a implementação, sendo que todos os aspectos (*front*, *back* e banco de dados) foram trabalhados simultaneamente, com testes de usabilidade a cada funcionalidade concluída, conforme descrito na Seção 3.10.

Como último passo, veio a formalização da documentação e da modelagem, conforme demonstrado no decorrer deste texto.

3 PROJETO PROPOSTO

Nesta seção, serão apresentados os detalhes por trás do funcionamento do MegaGame, além de seus modelos, instruções de uso e *proof of concept* (prova de conceito) visual.

3.1 Considerações iniciais

Conforme mencionado anteriormente, o sistema é feito em Django (com uso de SSR), consequentemente utilizando arquitetura baseada em componentes, trabalhando assim os conceitos da programação modular. Para o *front*, foi utilizado HTML/CSS/JavaScript conforme o padrão do desenvolvimento *web*. Para o banco de dados, foi empregado o SQLite, e o sistema também pode usar o PostgreSQL da plataforma Supabase quando for necessário o armazenamento em nuvem.

3.2 Perfis de usuário

Nesta seção, são descritos os perfis de usuário do MegaGame, com suas atividades e o que cada um pode acessar no sistema.

Os perfis correspondem aos atores dos casos de uso apresentados na Seção 3.4. Comprador e vendedor não são tipos diferentes de conta: qualquer usuário cadastrado pode comprar e vender com o mesmo perfil, e o papel depende apenas da atividade realizada. O gestor, por sua vez, usa uma conta da equipe do MegaGame, separada das contas dos clientes. Visitantes sem cadastro podem navegar pela página inicial, pelas categorias e pelos anúncios, mas precisam entrar no sistema para comprar, vender ou interagir com outros usuários.

3.2.1 GESTOR

O gestor é responsável pela administração da plataforma. Pelo *dashboard* administrativo, ele acompanha as vendas, as comissões recebidas e as despesas do MegaGame, que ele mesmo registra, e pode exportar esses dados em um relatório em PDF. Também cabe ao gestor definir as regras gerais do sistema, como a taxa de comissão cobrada sobre as vendas, os prazos de devolução e de abertura de disputas, o percentual de *cashback* em MegaCoins e a taxa fixa cobrada nas trocas.

Além disso, o gestor cria os cupons de desconto válidos em toda a plataforma e media as disputas abertas pelos usuários, decidindo cada caso a partir das mensagens e das evidências enviadas pelas partes. Por se tratar de uma conta da equipe, ele não

pode comprar, vender, dar lances em leilões nem propor trocas.

3.2.2 COMPRADOR

O comprador é o usuário que adquire produtos na plataforma. Ele pode pesquisar anúncios pela barra de busca, filtrá-los por preço, localização e avaliação e adicioná-los ao carrinho, finalizando o pedido com pagamento pelo Mercado Pago. Quando o vendedor permite, pode optar pela retirada em mãos em vez do envio. Depois da compra, acompanha o pedido pelo rastreio e pode cancelá-lo antes do envio ou solicitar a devolução dentro do prazo; caso o vendedor não resolva a solicitação, pode abrir uma disputa.

Além da compra direta, o comprador pode dar lances em leilões, acompanhar os leilões que lhe interessam e propor trocas nos anúncios que as aceitam. Ele também avalia os produtos recebidos e seus vendedores, aplica cupons de desconto e acumula MegaCoins, obtidas por *cashback*, pelas avaliações e pelo *check-in* diário na carteira, que podem abater parte do valor dos pedidos seguintes.

3.2.3 VENDEDOR

O vendedor é o usuário que anuncia produtos. Para receber os pagamentos, ele conecta sua própria conta do Mercado Pago ao MegaGame, e o valor de cada venda vai diretamente para essa conta, já descontada a comissão da plataforma. Ao criar um anúncio, define a categoria, a descrição, as imagens e as variações do produto, cada uma com preço e estoque próprios, e pode salvá-lo como rascunho para terminar depois. Uma vez publicado, o anúncio pode ser editado, retirado do ar temporariamente ou excluído. O vendedor também pode publicá-lo como leilão, com lance inicial, preço de reserva e opção de compra imediata, ou aceitar propostas de troca.

Os pedidos recebidos passam pelas etapas de confirmação, preparação e envio, com o código de rastreio informado pelo vendedor ou gerado pelo sistema. Nas compras com retirada em mãos, o envio dá lugar à etapa de produto pronto para retirada. O vendedor pode cancelar pedidos antes do envio, responde às solicitações de devolução e também pode levar o caso a uma disputa. Por fim, cria cupons de desconto para a própria loja e acompanha seu desempenho pelo *dashboard* de vendas.

3.3 Requisitos

Os requisitos do MegaGame são identificados por códigos, RF para os funcionais e RNF para os não funcionais, e classificados por prioridade em alta, média ou

baixa. Os requisitos funcionais são apresentados no Quadro 1.

Quadro 1 – Requisitos funcionais

Código	Requisito	Prioridade
RF1	O sistema deve permitir que o usuário efetue *login*.	Alta
RF2	O sistema deve permitir que novos usuários se cadastrem.	Alta
RF3	O sistema deve permitir a alteração dos dados de perfil após criação da conta de usuário.	Alta
RF4	O sistema deve registrar o histórico de pedidos, lances, trocas e movimentações de MegaCoins de cada usuário.	Média
RF5	O sistema deve possuir *dashboards* para o gestor e para os vendedores, para que eles possam acompanhar a parte financeira.	Alta
RF6	O sistema deve permitir que o usuário busque anúncios específicos através de uma barra de pesquisa, com filtros por preço, localização, avaliação e formato de venda.	Média
RF7	O sistema deve permitir a compra e venda de produtos através de anúncios.	Alta
RF8	O sistema deve permitir a edição e exclusão de anúncios depois que eles forem ao ar.	Alta
RF9	O sistema deve permitir a categorização de produtos.	Média
RF10	O sistema deve permitir a criação de variações para um produto, cada qual com suas informações específicas (nome, preço, estoque etc.).	Média
RF11	O sistema deve permitir que um usuário rastreie todo o fluxo de envio do produto que comprou antes de tê-lo em mãos.	Alta
RF12	O sistema deve permitir que um usuário cancele seu pedido antes do envio e solicite a devolução do produto em um certo período após a entrega.	Alta
RF13	O sistema deve permitir que seus usuários façam avaliações, tanto dos vendedores dos quais compraram quanto dos produtos que obtiveram.	Baixa

Código	Requisito	Prioridade
RF14	O sistema deve permitir que seus usuários naveguem pelas diferentes categorias cadastradas para ver todos os anúncios pertencentes a determinada categoria.	Baixa
RF15	O sistema deve permitir que o gestor altere a taxa de comissão recebida sobre as vendas e os demais parâmetros da plataforma, como os prazos de devolução e de disputa, o *cashback* e a taxa das trocas.	Alta
RF16	O sistema deve permitir que o comprador possa colocar produtos em um carrinho, para então finalizar o pedido quando estiver satisfeito com suas escolhas.	Alta
RF17	O sistema deve permitir que uma criação de anúncio interrompida possa ser retomada posteriormente.	Média
RF18	O sistema deve processar os pagamentos pelo Mercado Pago, repassando o valor de cada venda diretamente à conta do vendedor e retendo a comissão da plataforma.	Alta
RF19	O sistema deve permitir a retirada do produto em mãos, quando o vendedor aceitar essa opção.	Baixa
RF20	O sistema deve permitir a venda de produtos por leilão, com lance inicial, preço de reserva e opção de compra imediata, além do acompanhamento dos leilões pelos interessados.	Média
RF21	O sistema deve permitir que os usuários façam propostas e contrapropostas de troca nos anúncios que aceitem essa modalidade.	Média
RF22	O sistema deve permitir a abertura de disputas sobre solicitações de devolução, que serão decididas pelo gestor.	Alta
RF23	O sistema deve permitir a criação de cupons de desconto pelo gestor, válidos em toda a plataforma, e pelos vendedores, válidos em suas lojas.	Baixa
RF24	O sistema deve oferecer uma moeda virtual, a MegaCoin, obtida por *cashback*, por avaliações e por *check-in* diário, que possa abater parte do valor dos pedidos.	Baixa

Código	Requisito	Prioridade
RF25	O sistema deve permitir que o gestor registre as despesas da plataforma e exporte um relatório financeiro em PDF.	Média

Fonte: elaborado pelo próprio autor.

Já os não funcionais são apresentados no Quadro 2.

Quadro 2 – Requisitos não funcionais

Código	Requisito	Prioridade
RNF1	O sistema deve exibir o conteúdo principal de cada página em até 2,5 segundos.	Média
RNF2	O sistema deve suportar até 1000 usuários simultâneos mantendo o tempo de carregamento definido no RNF1.	Alta
RNF3	O sistema deve ser compatível com as versões atuais dos navegadores Chrome, Firefox, Edge e Safari, em computadores e celulares.	Alta
RNF4	O sistema deve garantir disponibilidade mínima de 99,5% ao longo de um ano.	Alta
RNF5	O sistema deve estar de acordo com a Lei Geral de Proteção de Dados Pessoais (LGPD).	Alta
RNF6	O sistema deve ser projetado para facilitar manutenção e eventuais atualizações, fazendo com que haja tempo mínimo de indisponibilidade em tais casos.	Média
RNF7	O sistema deve possuir documentação técnica completa.	Baixa
RNF8	A interface do sistema deve ser intuitiva, permitindo aprendizado suficiente para conduzir atividades em, no máximo, 1 hora.	Alta
RNF9	O sistema deve validar informações-chave do usuário durante um cadastro ou edição, como o *e-mail* ou a senha.	Alta
RNF10	O sistema deve tratar eventuais exceções para evitar brechas de segurança que possam causar prejuízo.	Alta
RNF11	O sistema deve possuir uma barra de navegação (nav).	Média

Código	Requisito	Prioridade
RNF12	As funcionalidades de compra, venda e negociação devem exigir uma sessão ativa na conta do usuário, mantendo a navegação pelos anúncios aberta a visitantes.	Alta
RNF13	O sistema deve estar de acordo com as práticas de responsividade e acessibilidade vigentes no mercado.	Baixa
RNF14	O sistema deve armazenar as senhas dos usuários apenas na forma de *hash*, nunca em texto puro.	Alta
RNF15	O sistema não deve armazenar dados de cartão dos usuários, deixando o processamento dos pagamentos inteiramente a cargo do Mercado Pago.	Alta

Fonte: elaborado pelo próprio autor.

Os valores numéricos dos requisitos não funcionais consideram um eventual ambiente de produção. O limite de 2,5 segundos do RNF1 segue a recomendação de Walton e Pollard (2025), que consideram bom esse tempo para a exibição do maior elemento visível de uma página. Já o RNF2 pressupõe o uso do PostgreSQL e de mais de um processo do servidor de aplicação, pois o SQLite, adotado na configuração atual (Seção 1.3), permite apenas uma escrita por vez em cada banco de dados (SQLite, [2026?b]).

3.4 Casos de uso

Nesta seção, serão apresentados os diagramas gerais de casos de uso e os casos de uso individuais em forma de documentação, para maiores detalhes.

A definição de casos de uso envolve uma sequência de interações entre um "ator" do sistema (seja este um usuário ou até mesmo outro sistema) e o sistema em si, gerando assim um resultado de valor para o ator em questão, conforme corroborado por Pressman e Maxim (2021). Eles ajudam a definir o comportamento desejado de um sistema em determinados cenários. No MegaGame, o Mercado Pago é um ator desse segundo tipo: ele aparece como ator secundário nos casos de uso que envolvem a conexão da conta do vendedor ou o pagamento de pedidos.

A relação completa dos casos de uso do MegaGame, com seus códigos e atores principais, é apresentada no Quadro 3.

Quadro 3 – Casos de uso do MegaGame

Código	**Caso de uso**	**Ator(es) principal(is)**
CSU01	Cadastrar Conta	Vendedor; Comprador

Código	Caso de uso	Ator(es) principal(is)
CSU02	Efetuar Login	Comprador; Vendedor
CSU03	Navegar pelas Categorias	Gestor; Comprador; Vendedor
CSU04	Pesquisar Produto	Gestor; Vendedor; Comprador
CSU05	Editar Perfil	Vendedor; Comprador
CSU06	Analisar o Dashboard Administrativo	Gestor
CSU07	Alterar Parâmetros da Plataforma	Gestor
CSU08	Adicionar Produto ao Carrinho	Comprador
CSU09	Fazer Pedido	Comprador
CSU10	Receber Produto	Comprador
CSU11	Avaliar Vendedor	Comprador
CSU12	Avaliar Produto	Comprador
CSU13	Cancelar Pedido	Comprador; Vendedor
CSU14	Analisar Dashboard de Vendas	Vendedor
CSU15	Ver Pedidos Recebidos	Vendedor
CSU16	Preparar Entrega	Vendedor
CSU17	Publicar Anúncio	Vendedor
CSU18	Editar Anúncio	Vendedor
CSU19	Retirar Anúncio do Ar	Vendedor
CSU20	Deletar Anúncio	Vendedor
CSU21	Conectar Conta do Mercado Pago	Vendedor
CSU22	Rastrear Pedido	Comprador
CSU23	Solicitar Devolução	Comprador
CSU24	Responder Solicitação de Devolução	Vendedor
CSU25	Abrir Disputa	Comprador; Vendedor
CSU26	Resolver Disputa	Gestor
CSU27	Criar Leilão	Vendedor
CSU28	Gerenciar Leilão	Vendedor
CSU29	Dar Lance	Comprador
CSU30	Comprar Agora	Comprador
CSU31	Acompanhar Leilão	Comprador
CSU32	Propor Troca	Comprador
CSU33	Responder Proposta de Troca	Vendedor; Comprador
CSU34	Gerenciar Cupons	Gestor; Vendedor
CSU35	Salvar Cupom	Comprador
CSU36	Consultar Carteira de MegaCoins	Comprador

Código	Caso de uso	Ator(es) principal(is)
CSU37	Fazer Check-in Diário	Comprador
CSU38	Registrar Despesa	Gestor
CSU39	Exportar Relatório Financeiro	Gestor

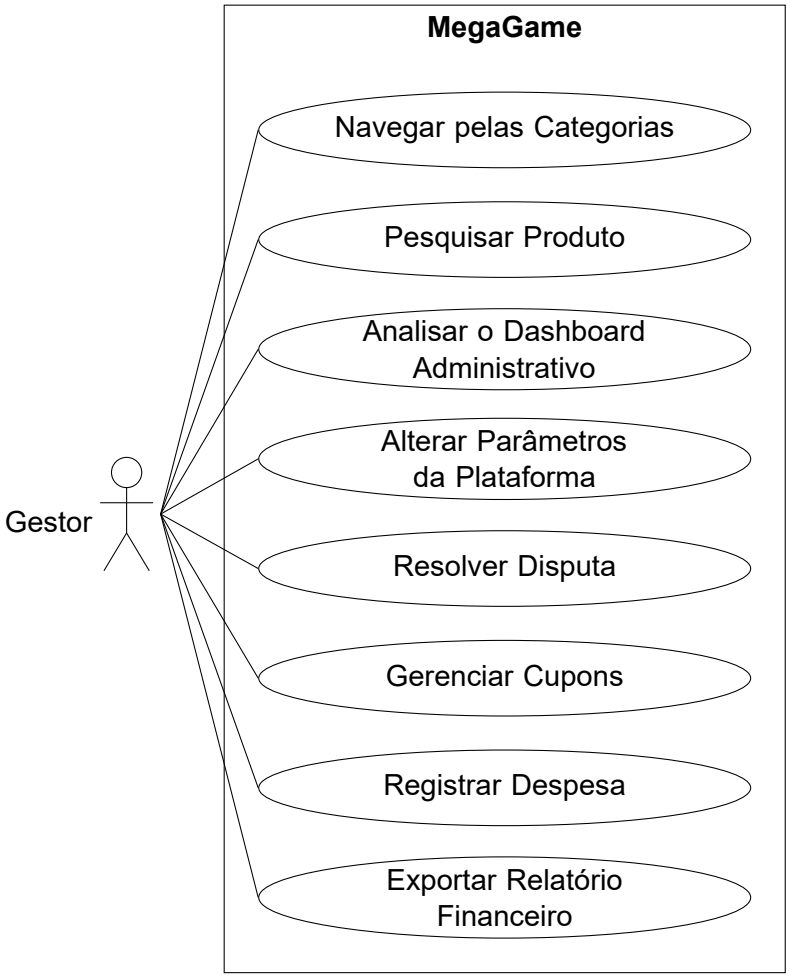
Fonte: elaborado pelo próprio autor.

3.4.1 DIAGRAMAS DE CASOS DE USO

Os diagramas de casos de uso estão organizados por ator, e o do gestor é apresentado na Figura 7. Como o comprador e o vendedor participam de muitos casos, cada um tem dois diagramas. As compras e o pós-venda do comprador estão na Figura 8, e seus leilões, trocas e recompensas, na Figura 9. Já a conta, os anúncios e os leilões do vendedor aparecem na Figura 10, e suas vendas e o pós-venda, na Figura 11.

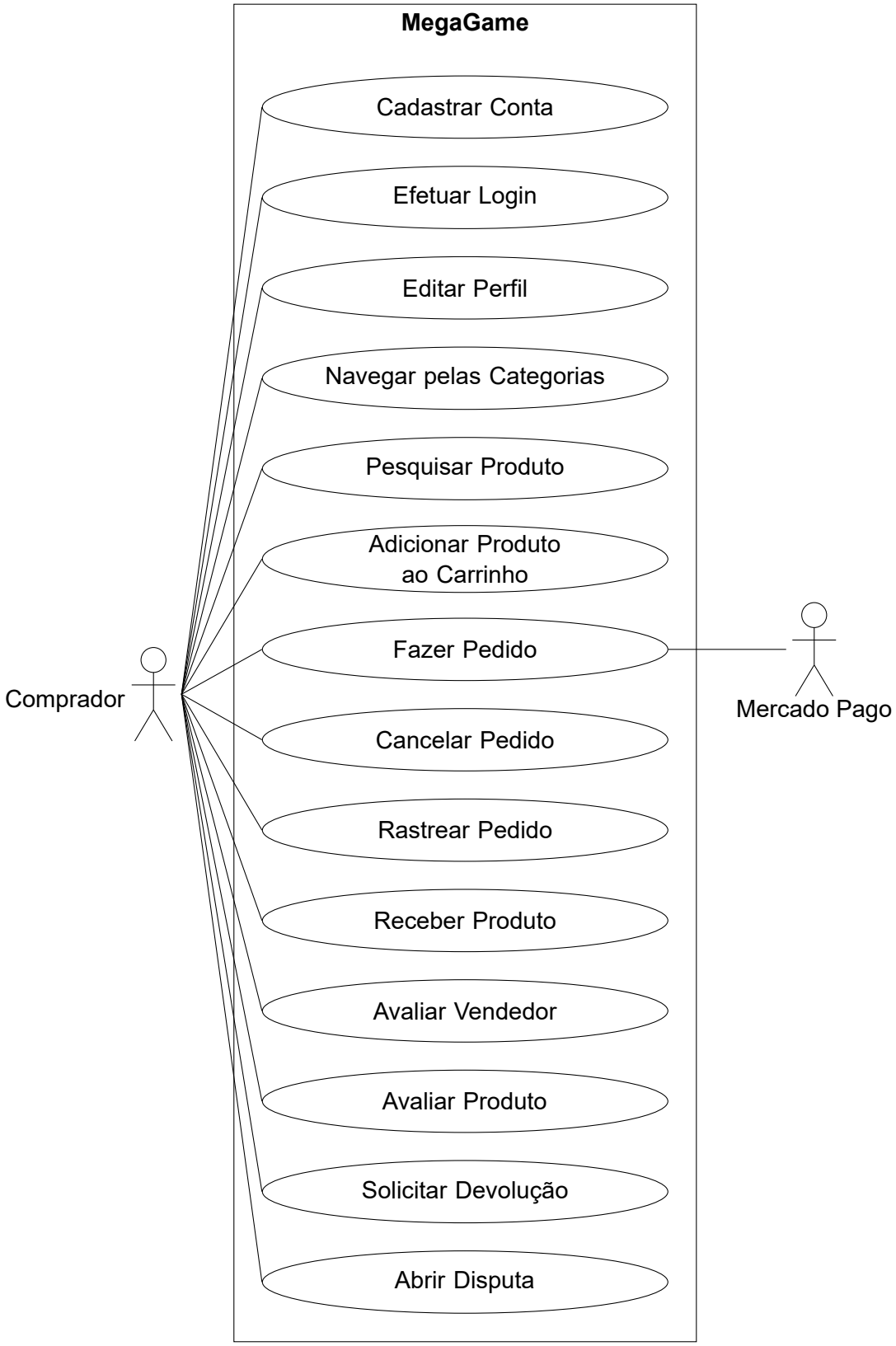
Os diagramas mostram apenas as associações entre os atores e os casos de uso. A ordem em que os casos acontecem está nos campos de pré-condições e pós-condições dos quadros da próxima subseção.

Figura 7 – Diagrama de casos de uso do gestor



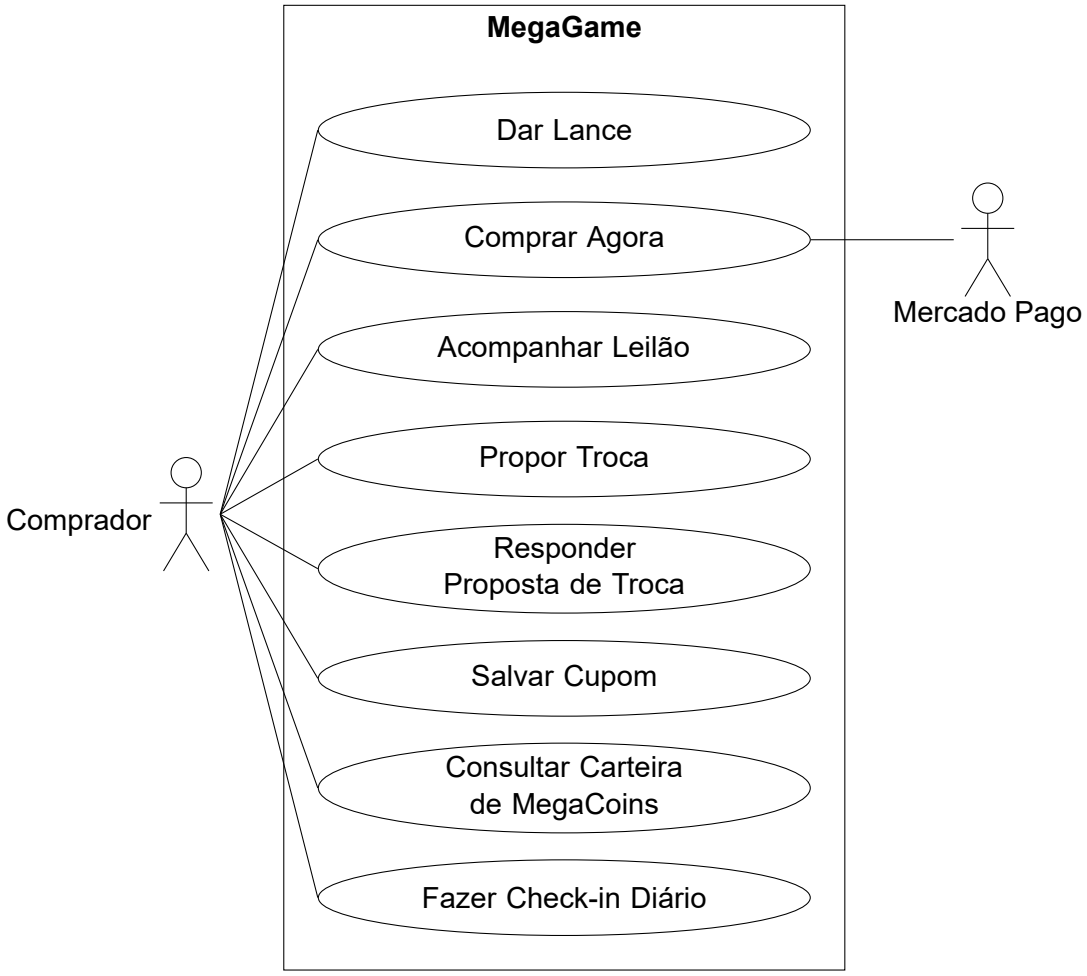
Fonte: elaborado pelo próprio autor.

Figura 8 – Diagrama de casos de uso do comprador: compras e pós-venda



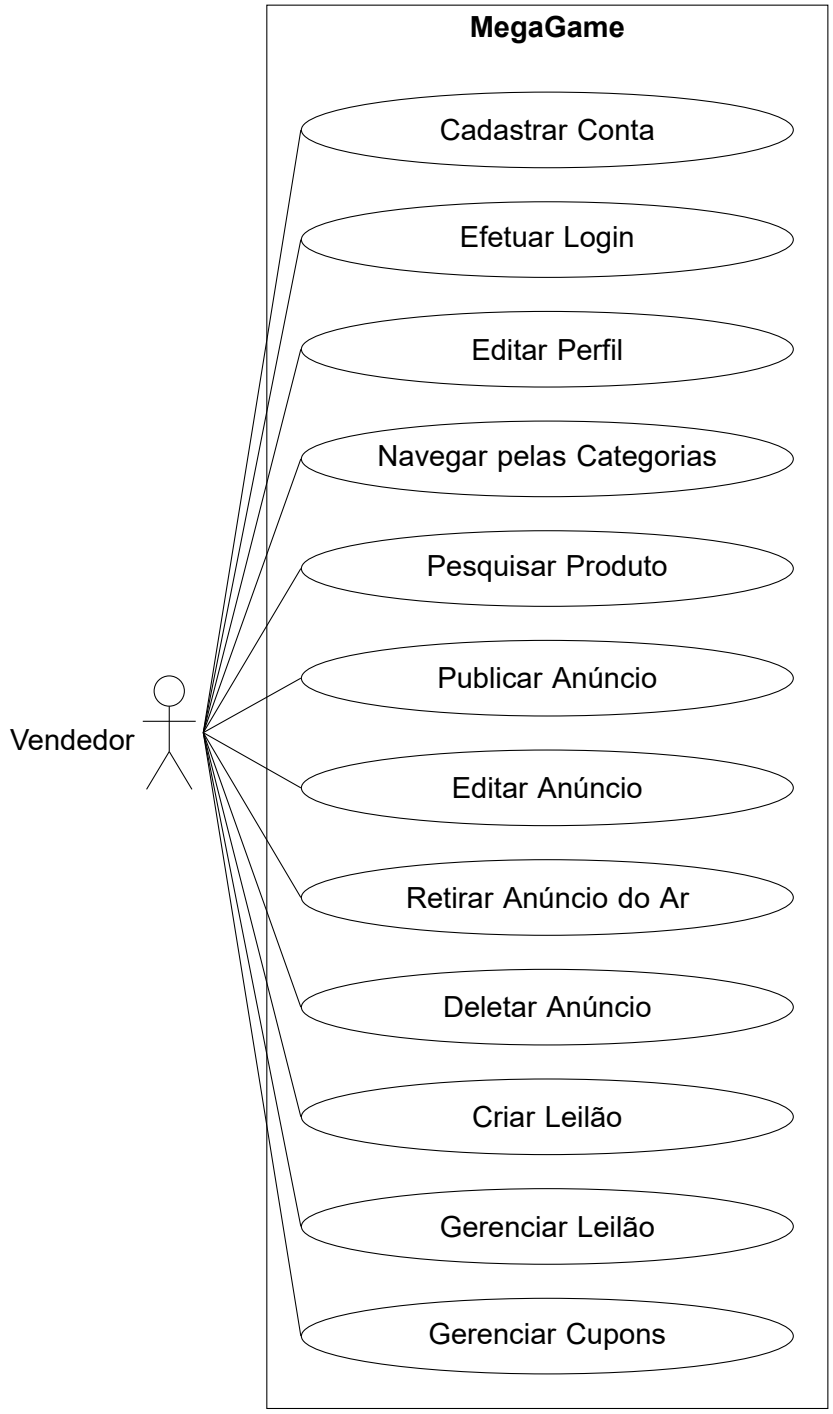
Fonte: elaborado pelo próprio autor.

Figura 9 – Diagrama de casos de uso do comprador: leilões, trocas e recompensas



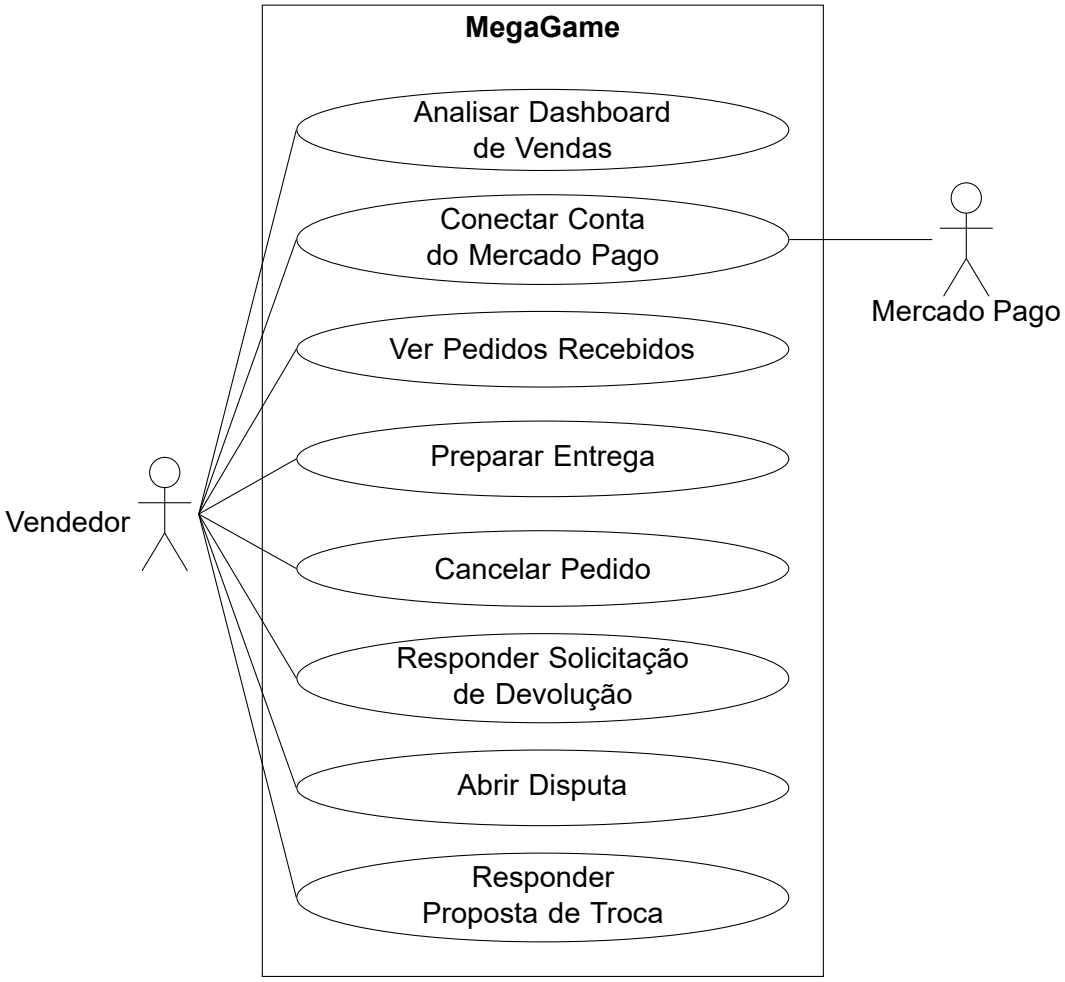
Fonte: elaborado pelo próprio autor.

Figura 10 – Diagrama de casos de uso do vendedor: conta, anúncios e leilões



Fonte: elaborado pelo próprio autor.

Figura 11 – Diagrama de casos de uso do vendedor: vendas e pós-venda



Fonte: elaborado pelo próprio autor.

3.4.2 DESCRIÇÕES DE CASOS DE USO

Os Quadros 4 a 42 descrevem os casos de uso mostrados na subseção anterior, porém sob uma perspectiva individual e detalhada.

Quadro 4 – Caso de uso: Cadastrar conta

Nome do Caso de Uso	CSU01 – Cadastrar Conta
Ator Principal	Vendedor; Comprador
Ator Secundário	
Resumo	Cadastrar uma nova conta de usuário no sistema.
Pré-Condições	
Pós-Condições	Efetuar Login

Fonte: elaborado pelo próprio autor.

Quadro 5 – Caso de uso: Efetuar login

Nome do Caso de Uso	CSU02 – Efetuar Login
Ator Principal	Comprador; Vendedor
Ator Secundário	
Resumo	Efetuar login em uma conta de usuário cadastrada no sistema.
Pré-Condições	Cadastrar Conta
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 6 – Caso de uso: Navegar pelas categorias

Nome do Caso de Uso	CSU03 – Navegar pelas Categorias
Ator Principal	Gestor; Comprador; Vendedor
Ator Secundário	
Resumo	Navegar pela aba de “Categorias” e ver os diferentes produtos de cada categoria.
Pré-Condições	
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 7 – Caso de uso: Pesquisar produto

Nome do Caso de Uso	CSU04 – Pesquisar Produto
Ator Principal	Gestor; Vendedor; Comprador
Ator Secundário	
Resumo	Pesquisar um produto na barra de busca do sistema, com filtros por preço, localização, avaliação e formato de venda.
Pré-Condições	
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 8 – Caso de uso: Editar perfil

Nome do Caso de Uso	CSU05 – Editar Perfil
Ator Principal	Vendedor; Comprador

Ator Secundário	
Resumo	Editar as informações inseridas no perfil (nome, senha, localização etc.).
Pré-Condições	
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 9 – Caso de uso: Analisar o *dashboard* administrativo

Nome do Caso de Uso	CSU06 – Analisar o Dashboard Administrativo
Ator Principal	Gestor
Ator Secundário	
Resumo	Observar o painel administrativo para ver os relatórios do sistema e estatísticas de venda.
Pré-Condições	
Pós-Condições	Alterar Parâmetros da Plataforma; Registrar Despesa; Exportar Relatório Financeiro

Fonte: elaborado pelo próprio autor.

Quadro 10 – Caso de uso: Alterar parâmetros da plataforma

Nome do Caso de Uso	CSU07 – Alterar Parâmetros da Plataforma
Ator Principal	Gestor
Ator Secundário	
Resumo	Alterar a taxa de comissão cobrada sobre as vendas e os demais parâmetros da plataforma, como os prazos de devolução e de disputa, o *cashback* e a taxa das trocas.
Pré-Condições	Analisar o Dashboard Administrativo
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 11 – Caso de uso: Adicionar produto ao carrinho

Nome do Caso de Uso	CSU08 – Adicionar Produto ao Carrinho
Ator Principal	Comprador
Ator Secundário	
Resumo	O comprador adiciona um produto que gostaria de comprar ao seu carrinho.
Pré-Condições	
Pós-Condições	Fazer Pedido

Fonte: elaborado pelo próprio autor.

Quadro 12 – Caso de uso: Fazer pedido

Nome do Caso de Uso	CSU09 – Fazer Pedido
Ator Principal	Comprador
Ator Secundário	Mercado Pago
Resumo	O usuário finaliza o pedido com todos os itens presentes em seu carrinho. Antes do pagamento, pode aplicar cupons, usar MegaCoins e escolher a retirada em mãos, quando o vendedor aceita essa opção.
Pré-Condições	Adicionar Produto ao Carrinho
Pós-Condições	Cancelar Pedido; Rastrear Pedido; Receber Produto

Fonte: elaborado pelo próprio autor.

Quadro 13 – Caso de uso: Receber produto

Nome do Caso de Uso	CSU10 – Receber Produto
Ator Principal	Comprador
Ator Secundário	Vendedor
Resumo	Após a preparação da entrega, o comprador recebe o produto pelo envio ou o retira em mãos com o vendedor.
Pré-Condições	Fazer Pedido
Pós-Condições	Avaliar Vendedor; Avaliar Produto; Solicitar Devolução

Fonte: elaborado pelo próprio autor.

Quadro 14 – Caso de uso: Avaliar vendedor

Nome do Caso de Uso	CSU11 – Avaliar Vendedor
Ator Principal	Comprador
Ator Secundário	
Resumo	Após receber um produto que comprou, o usuário avalia o vendedor.
Pré-Condições	Receber Produto
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 15 – Caso de uso: Avaliar produto

Nome do Caso de Uso	CSU12 – Avaliar Produto
Ator Principal	Comprador
Ator Secundário	
Resumo	O usuário avalia um produto que comprou após recebê-lo.
Pré-Condições	Receber Produto
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 16 – Caso de uso: Cancelar pedido

Nome do Caso de Uso	CSU13 – Cancelar Pedido
Ator Principal	Comprador; Vendedor
Ator Secundário	
Resumo	O pedido é cancelado antes do envio: pelo comprador, enquanto aguarda pagamento ou já está pago; ou pelo vendedor, enquanto está pago, confirmado ou em preparação.
Pré-Condições	Fazer Pedido; Ver Pedidos Recebidos
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 17 – Caso de uso: Analisar *dashboard* de vendas

Nome do Caso de Uso	CSU14 – Analisar Dashboard de Vendas
Ator Principal	Vendedor

Ator Secundário	
Resumo	O vendedor analisa seu *dashboard* de vendas para ver suas estatísticas e relatórios.
Pré-Condições	
Pós-Condições	Ver Pedidos Recebidos; Conectar Conta do Mercado Pago

Fonte: elaborado pelo próprio autor.

Quadro 18 – Caso de uso: Ver pedidos recebidos

Nome do Caso de Uso	CSU15 – Ver Pedidos Recebidos
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor vê os pedidos que recebeu de compradores.
Pré-Condições	Analisar Dashboard de Vendas
Pós-Condições	Cancelar Pedido; Preparar Entrega

Fonte: elaborado pelo próprio autor.

Quadro 19 – Caso de uso: Preparar entrega

Nome do Caso de Uso	CSU16 – Preparar Entrega
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor confirma o pedido, prepara o produto e o envia, informando ou gerando o código de rastreio. Nos pedidos com retirada em mãos, marca o produto como pronto para retirada.
Pré-Condições	Ver Pedidos Recebidos
Pós-Condições	Rastrear Pedido; Receber Produto

Fonte: elaborado pelo próprio autor.

Quadro 20 – Caso de uso: Publicar anúncio

Nome do Caso de Uso	CSU17 – Publicar Anúncio
Ator Principal	Vendedor
Ator Secundário	

Resumo	O vendedor publica o anúncio de um produto, com categoria, descrição, imagens e variações. Um anúncio interrompido fica salvo como rascunho e pode ser retomado depois.
Pré-Condições	
Pós-Condições	Editar Anúncio; Retirar Anúncio do Ar; Deletar Anúncio; Criar Leilão

Fonte: elaborado pelo próprio autor.

Quadro 21 – Caso de uso: Editar anúncio

Nome do Caso de Uso	CSU18 – Editar Anúncio
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor edita as informações de um anúncio após sua publicação inicial.
Pré-Condições	Publicar Anúncio
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 22 – Caso de uso: Retirar anúncio do ar

Nome do Caso de Uso	CSU19 – Retirar Anúncio do Ar
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor retira o anúncio do ar temporariamente. Nesse estado, ele ainda pode editá-lo e depois republicá-lo.
Pré-Condições	Publicar Anúncio
Pós-Condições	Publicar Anúncio; Editar Anúncio

Fonte: elaborado pelo próprio autor.

Quadro 23 – Caso de uso: Deletar anúncio

Nome do Caso de Uso	CSU20 – Deletar Anúncio
Ator Principal	Vendedor
Ator Secundário	

Resumo	O vendedor deleta um anúncio (ao menos para si e outros usuários comuns; os registros ainda ficarão no sistema por garantia).
Pré-Condições	Publicar Anúncio
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 24 – Caso de uso: Conectar conta do Mercado Pago

Nome do Caso de Uso	CSU21 – Conectar Conta do Mercado Pago
Ator Principal	Vendedor
Ator Secundário	Mercado Pago
Resumo	O vendedor autoriza o MegaGame a acessar sua conta do Mercado Pago, que passa a receber diretamente o valor das vendas, já descontada a comissão da plataforma.
Pré-Condições	Analisar Dashboard de Vendas
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 25 – Caso de uso: Rastrear pedido

Nome do Caso de Uso	CSU22 – Rastrear Pedido
Ator Principal	Comprador
Ator Secundário	
Resumo	O comprador acompanha as etapas de envio de um pedido a partir do código de rastreio informado ou gerado pelo vendedor.
Pré-Condições	Preparar Entrega
Pós-Condições	Receber Produto

Fonte: elaborado pelo próprio autor.

Quadro 26 – Caso de uso: Solicitar devolução

Nome do Caso de Uso	CSU23 – Solicitar Devolução
Ator Principal	Comprador
Ator Secundário	Vendedor
Resumo	Dentro do prazo de devolução definido pelo gestor, o comprador pede a devolução de um produto entregue, informando o motivo e podendo anexar imagens.
Pré-Condições	Receber Produto
Pós-Condições	Responder Solicitação de Devolução; Abrir Disputa

Fonte: elaborado pelo próprio autor.

Quadro 27 – Caso de uso: Responder solicitação de devolução

Nome do Caso de Uso	CSU24 – Responder Solicitação de Devolução
Ator Principal	Vendedor
Ator Secundário	Comprador
Resumo	O vendedor aceita a devolução e, ao receber o produto de volta, confirma o recebimento, o que devolve o item ao estoque. Ele também pode reembolsar o comprador sem pedir o produto de volta.
Pré-Condições	Solicitar Devolução
Pós-Condições	Abrir Disputa

Fonte: elaborado pelo próprio autor.

Quadro 28 – Caso de uso: Abrir disputa

Nome do Caso de Uso	CSU25 – Abrir Disputa
Ator Principal	Comprador; Vendedor
Ator Secundário	Gestor

Resumo	Uma solicitação de devolução sem acordo é levada ao gestor, com uma descrição e imagens como evidência. O vendedor pode contestar a solicitação assim que a recebe. O comprador abre a disputa depois que termina o prazo de resposta do vendedor, ou contesta um cancelamento sem devolução.
Pré-Condições	Solicitar Devolução; Responder Solicitação de Devolução
Pós-Condições	Resolver Disputa

Fonte: elaborado pelo próprio autor.

Quadro 29 – Caso de uso: Resolver disputa

Nome do Caso de Uso	CSU26 – Resolver Disputa
Ator Principal	Gestor
Ator Secundário	Comprador; Vendedor
Resumo	O gestor analisa as mensagens e evidências das duas partes e decide a disputa a favor do comprador, com ou sem a devolução do produto, ou a favor do vendedor.
Pré-Condições	Abrir Disputa
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 30 – Caso de uso: Criar leilão

Nome do Caso de Uso	CSU27 – Criar Leilão
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor coloca um anúncio em leilão e define o lance inicial e a duração. Se quiser, define também um preço de reserva e um preço de compra imediata.
Pré-Condições	Publicar Anúncio

Pós-Condições	Gerenciar Leilão; Dar Lance; Comprar Agora; Acompanhar Leilão

Fonte: elaborado pelo próprio autor.

Quadro 31 – Caso de uso: Gerenciar leilão

Nome do Caso de Uso	CSU28 – Gerenciar Leilão
Ator Principal	Vendedor
Ator Secundário	Comprador
Resumo	O vendedor encerra o leilão antes do prazo, cancela a venda quando o vencedor não paga ou relista o item. Também pode fazer uma oferta de segunda chance a outro participante, que a aceita ou recusa.
Pré-Condições	Criar Leilão
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 32 – Caso de uso: Dar lance

Nome do Caso de Uso	CSU29 – Dar Lance
Ator Principal	Comprador
Ator Secundário	
Resumo	O comprador informa o valor máximo que aceita pagar, e o sistema cobre os lances seguintes de forma automática até esse limite. O lance pode ser retirado por um dos motivos aceitos pelo sistema, como erro de digitação.
Pré-Condições	Criar Leilão
Pós-Condições	Rastrear Pedido; Receber Produto

Fonte: elaborado pelo próprio autor.

Quadro 33 – Caso de uso: Comprar agora

Nome do Caso de Uso	CSU30 – Comprar Agora
Ator Principal	Comprador
Ator Secundário	Mercado Pago

Resumo	O comprador encerra o leilão ao pagar o preço de compra imediata, opção disponível enquanto não houver lances ou enquanto o preço de reserva não for atingido.
Pré-Condições	Criar Leilão
Pós-Condições	Rastrear Pedido; Receber Produto

Fonte: elaborado pelo próprio autor.

Quadro 34 – Caso de uso: Acompanhar leilão

Nome do Caso de Uso	CSU31 – Acompanhar Leilão
Ator Principal	Comprador
Ator Secundário	
Resumo	O comprador marca um leilão para acompanhá-lo e consulta, em uma página própria, os leilões acompanhados, aqueles em que deu lances e as ofertas de segunda chance recebidas.
Pré-Condições	Criar Leilão
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 35 – Caso de uso: Propor troca

Nome do Caso de Uso	CSU32 – Propor Troca
Ator Principal	Comprador
Ator Secundário	Vendedor
Resumo	O comprador oferece produtos do seu próprio estoque por um anúncio que aceita trocas, podendo incluir uma volta em dinheiro. O anunciante tem um prazo para responder.
Pré-Condições	Publicar Anúncio
Pós-Condições	Responder Proposta de Troca

Fonte: elaborado pelo próprio autor.

Quadro 36 – Caso de uso: Responder proposta de troca

Nome do Caso de Uso	CSU33 – Responder Proposta de Troca
Ator Principal	Vendedor; Comprador
Ator Secundário	
Resumo	Quem aguarda a resposta aceita, recusa ou faz uma contraproposta, que passa para o outro lado; o proponente também pode retirar a proposta. Uma troca aceita gera os pedidos de envio dos produtos, com a taxa definida pelo gestor.
Pré-Condições	Propor Troca
Pós-Condições	Rastrear Pedido; Receber Produto

Fonte: elaborado pelo próprio autor.

Quadro 37 – Caso de uso: Gerenciar cupons

Nome do Caso de Uso	CSU34 – Gerenciar Cupons
Ator Principal	Gestor; Vendedor
Ator Secundário	
Resumo	O gestor cria cupons válidos em toda a plataforma, e o vendedor, cupons da sua loja, com desconto percentual ou de valor fixo, validade, valor mínimo de compra e limite de usos. Os cupons podem ser editados, encerrados, reativados ou excluídos.
Pré-Condições	
Pós-Condições	Salvar Cupom

Fonte: elaborado pelo próprio autor.

Quadro 38 – Caso de uso: Salvar cupom

Nome do Caso de Uso	CSU35 – Salvar Cupom
Ator Principal	Comprador
Ator Secundário	
Resumo	O comprador adiciona um cupom aos seus cupons por meio do código, para usá-lo ao fazer um pedido.

Pré-Condições	Gerenciar Cupons
Pós-Condições	Fazer Pedido

Fonte: elaborado pelo próprio autor.

Quadro 39 – Caso de uso: Consultar carteira de MegaCoins

Nome do Caso de Uso	CSU36 – Consultar Carteira de MegaCoins
Ator Principal	Comprador
Ator Secundário	
Resumo	O comprador consulta seu saldo de MegaCoins e o valor correspondente em reais, seu nível, o *cashback* previsto para os pedidos em andamento e o histórico de movimentações.
Pré-Condições	
Pós-Condições	Fazer Check-in Diário; Fazer Pedido

Fonte: elaborado pelo próprio autor.

Quadro 40 – Caso de uso: Fazer *check-in* diário

Nome do Caso de Uso	CSU37 – Fazer Check-in Diário
Ator Principal	Comprador
Ator Secundário	
Resumo	Uma vez por dia, o comprador faz o *check-in* na carteira e recebe MegaCoins, em quantidade que varia conforme o dia da sequência.
Pré-Condições	Consultar Carteira de MegaCoins
Pós-Condições	

Fonte: elaborado pelo próprio autor.

Quadro 41 – Caso de uso: Registrar despesa

Nome do Caso de Uso	CSU38 – Registrar Despesa
Ator Principal	Gestor
Ator Secundário	

Resumo	O gestor lança, edita ou exclui as despesas da plataforma, que entram no resultado financeiro exibido no *dashboard* administrativo.
Pré-Condições	Analisar o Dashboard Administrativo
Pós-Condições	Exportar Relatório Financeiro

Fonte: elaborado pelo próprio autor.

Quadro 42 – Caso de uso: Exportar relatório financeiro

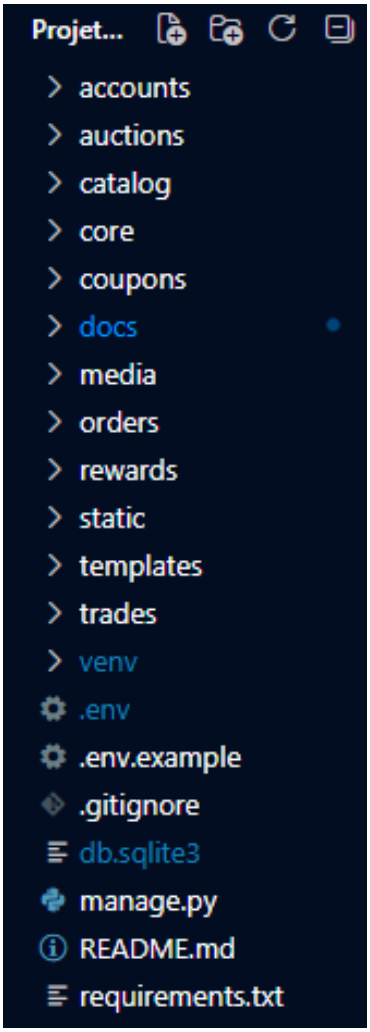
Nome do Caso de Uso	CSU39 – Exportar Relatório Financeiro
Ator Principal	Gestor
Ator Secundário	
Resumo	O gestor gera um arquivo PDF com os dados financeiros do *dashboard* administrativo.
Pré-Condições	Analisar o Dashboard Administrativo
Pós-Condições	

Fonte: elaborado pelo próprio autor.

3.5 Arquitetura

O MegaGame segue a organização padrão do Django, que separa o projeto, definido principalmente por um módulo de configurações, dos *apps*, pacotes Python que reúnem um conjunto de funcionalidades (Django Software Foundation, [2026?a]). A Figura 12 mostra os arquivos e as pastas na raiz do projeto.

Figura 12 – Estrutura de arquivos na raiz do projeto



Fonte: elaborado pelo próprio autor.

Além das pastas dos *apps*, a raiz contém a pasta templates, com os arquivos HTML das páginas organizados em subpastas por *app*; a static, com as folhas de estilo, os *scripts* JavaScript e as imagens fixas do site; a media, onde ficam os arquivos enviados pelos usuários, como as fotos dos anúncios; e a docs, com esta documentação. O manage.py executa os comandos administrativos do Django, o requirements.txt lista as bibliotecas de que o projeto depende e o db.sqlite3 é o próprio banco de dados. Já as chaves e credenciais, como as do Mercado Pago e a chave secreta do Django, ficam no arquivo .env, que não entra no controle de versão; o .env.example traz apenas o modelo desse arquivo, sem os valores.

A maioria dos *apps* tem a mesma estrutura de arquivos. O models.py define os modelos, que dão origem às tabelas do banco; o views.py reúne as funções que tratam cada requisição; o urls.py associa as rotas a essas funções; o forms.py declara os formulários; o admin.py registra os modelos no painel administrativo; e a pasta migrations guarda as *migrations* geradas a partir dos modelos. Nos *apps* com regras de negócio mais extensas, como os de leilões, trocas, cupons e MegaCoins, essa

lógica fica separada em um arquivo services.py. A função de cada *app* é a seguinte:

a) core: pacote do projeto criado pelo próprio Django, com as configurações gerais e o roteamento principal das URLs;
b) accounts: cadastro e perfil dos usuários, avaliação de vendedores e conexão da conta do Mercado Pago;
c) catalog: categorias, anúncios com suas variações e imagens, avaliação de produtos, página inicial e busca;
d) orders: carrinho, pedidos e pagamento pelo Mercado Pago; entrega, rastreio, devoluções e disputas; configurações da plataforma, comissões e despesas; e os *dashboards*, com o relatório em PDF;
e) rewards: carteira de MegaCoins, com o *cashback*, a recompensa por avaliações e o *check-in* diário;
f) coupons: criação e resgate dos cupons de desconto da plataforma e das lojas;
g) auctions: leilões, lances, acompanhamento e ofertas de segunda chance;
h) trades: propostas e contrapropostas de troca e o histórico de cada negociação.

3.6 Projeto de dados

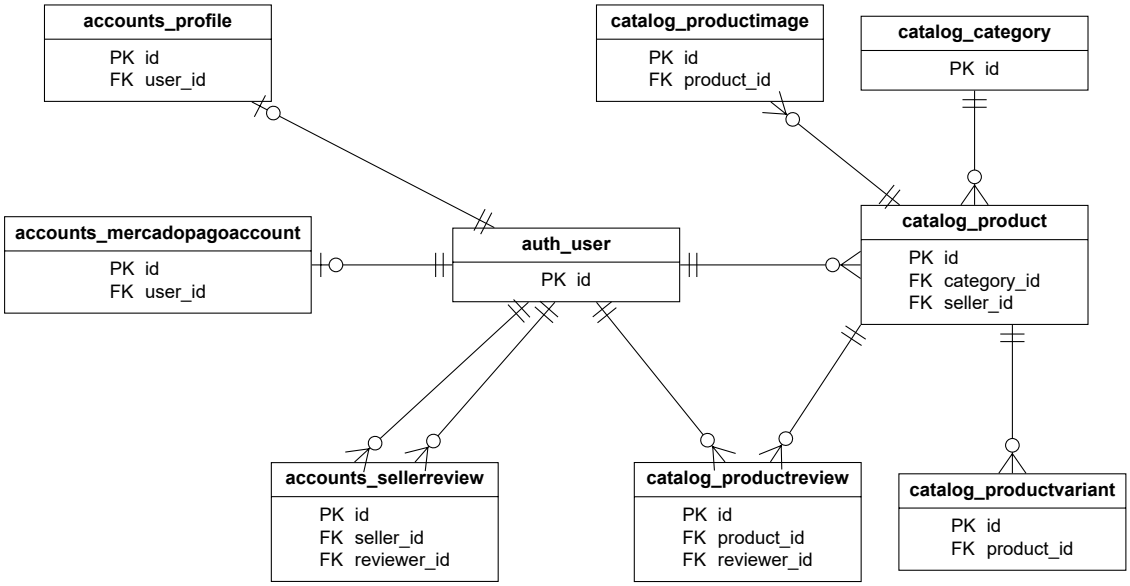
A principal fonte acadêmica a respeito do tema “banco de dados” é Coronel e suas obras, com vários coautores ao longo dos anos, tais como Morris e Rob. Segundo Coronel e Morris (2022), o *design* de banco de dados compreende as atividades de planejamento da estrutura que armazenará e gerenciará os dados dos usuários do sistema. Para os autores, um banco que atenda a todos os requisitos não surge de forma espontânea: sua estrutura precisa ser planejada com cuidado, pois nem mesmo um bom SGBD compensa o desempenho de um banco de dados mal projetado.

No MegaGame, as tabelas são definidas pelos modelos de cada *app* e criadas no SQLite pelo processo de *migration*, descrito na Seção 2.2.3. Os diagramas entidade-relacionamento (DER) foram elaborados a partir desses modelos, com a notação pé de galinha (*crow’s foot*) adotada por Coronel e Morris (2022). Como os *apps* criam 30 tabelas e quase todas se ligam à tabela de usuários do Django, o DER foi dividido em quatro módulos: usuários e catálogo (Figura 13), pedidos e pós-venda (Figura 14), leilões e trocas (Figura 15) e cupons, MegaCoins e gestão (Figura 16).

Cada tabela mostra apenas sua chave primária (*primary key* – PK) e suas chaves estrangeiras (*foreign key* – FK); os demais campos estão nos dicionários de dados da próxima subseção. As tabelas de borda tracejada pertencem a outro módulo e aparecem só para indicar o relacionamento, às vezes repetidas no mesmo diagrama para

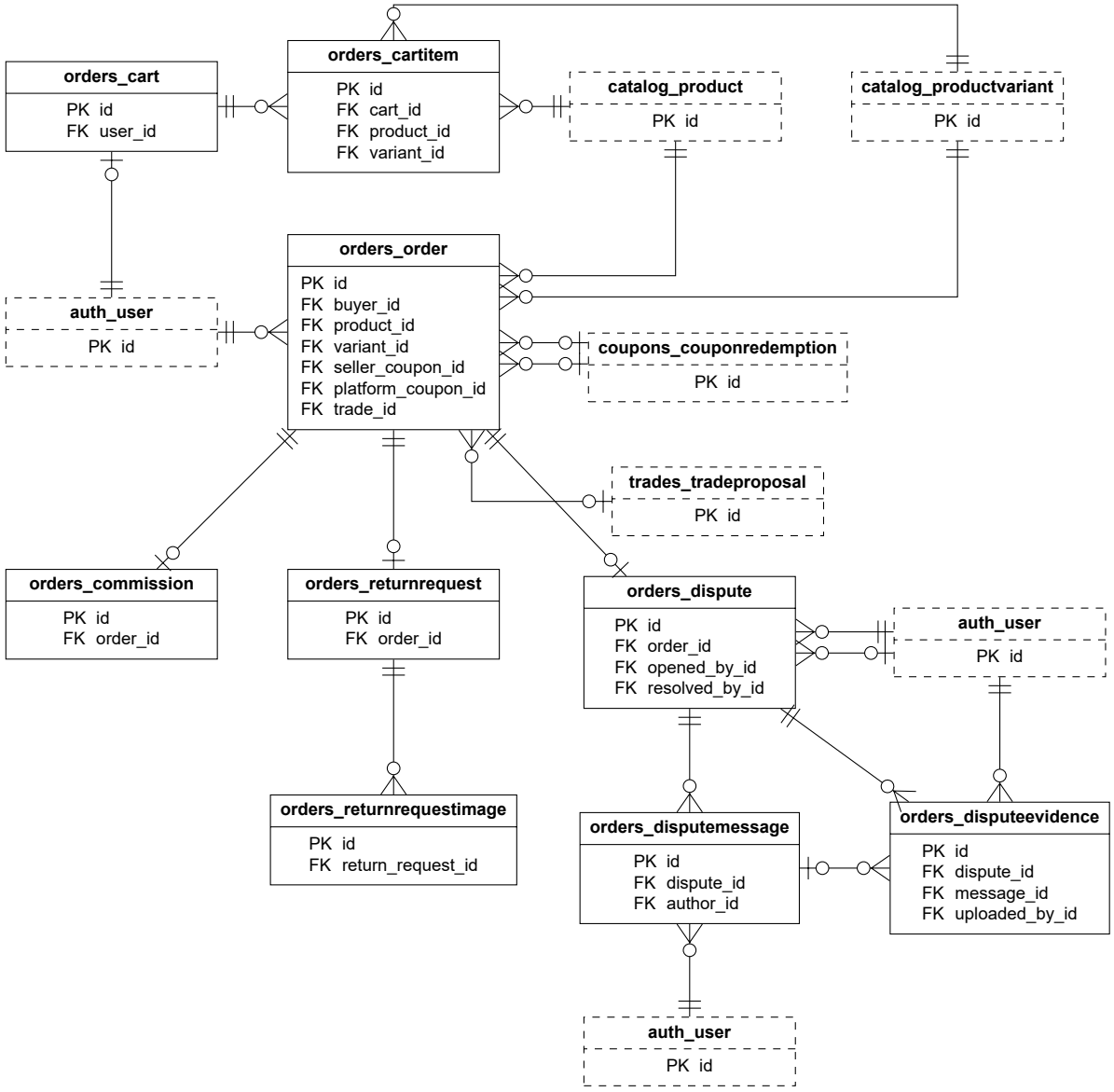
evitar o cruzamento de linhas. As tabelas internas do Django, como as de sessões, permissões e registro do painel administrativo, foram omitidas. A de usuários (auth_user), embora também venha do Django, foi mantida, porque é referenciada por quase todo o sistema.

Figura 13 – DER: usuários e catálogo



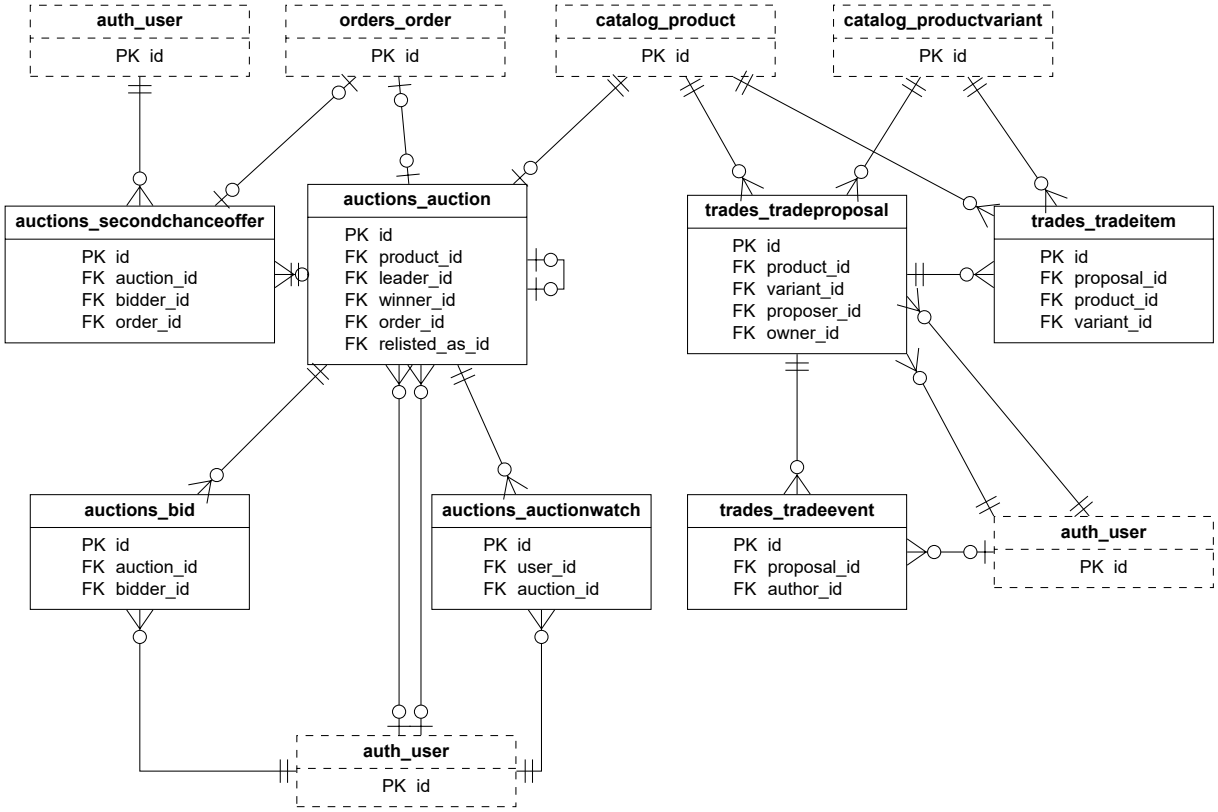
Fonte: elaborado pelo próprio autor.

Figura 14 – DER: pedidos e pós-venda



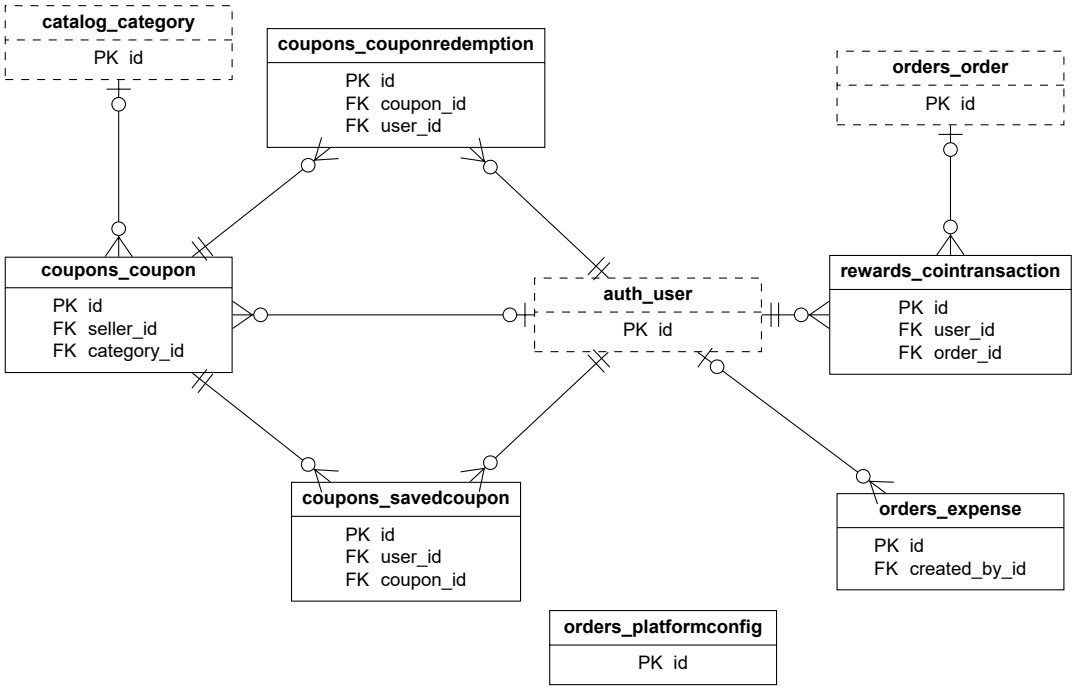
Fonte: elaborado pelo próprio autor.

Figura 15 – DER: leilões e trocas



Fonte: elaborado pelo próprio autor.

Figura 16 – DER: cupons, MegaCoins e gestão



Fonte: elaborado pelo próprio autor.

3.6.1 DICIONÁRIO DE DADOS

Os Quadros 43 a 73 apresentam os dicionários de dados das tabelas do Mega-Game, na mesma ordem dos módulos dos DERs. Nos quadros, a coluna “Tipo” traz o tipo da coluna no SQLite, conforme criado pelo Django; a coluna “Obrigatório?” indica se o campo deve sempre ter um valor (SIM) ou se pode ficar nulo (NÃO); e a coluna “Chave?” indica se o campo é uma chave primária (PK) ou uma chave estrangeira (FK), caso em que a tabela referenciada aparece entre parênteses na descrição. As siglas ID designam identificadores.

Quadro 43 – Dicionário de dados: Usuários (auth_user)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do usuário	integer	SIM	PK
password	Senha do usuário, armazenada como *hash*	varchar(128)	SIM	
last_login	Data do último *login* do usuário	datetime	NÃO	
is_superuser	O usuário tem todas as permissões do sistema?	bool	SIM	
username	Nome do usuário dentro da plataforma	varchar(150)	SIM	
first_name	Nome real do usuário	varchar(150)	SIM	
last_name	Sobrenome do usuário	varchar(150)	SIM	
email	E-mail do usuário	varchar(254)	SIM	
is_staff	O usuário faz parte da equipe por trás do sistema (gestor)?	bool	SIM	
is_active	O usuário é ativo?	bool	SIM	
date_joined	Data na qual o usuário criou sua conta	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 44 – Dicionário de dados: Perfis de usuário (accounts_profile)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do perfil	integer	SIM	PK
user_id	Usuário dono do perfil (auth_user)	integer	SIM	FK
city	Cidade do usuário	varchar(100)	SIM	
uf	Sigla do estado do usuário	varchar(2)	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 45 – Dicionário de dados: Contas do Mercado Pago (accounts_ mercadopagoaccount)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da conta conectada	integer	SIM	PK
user_id	Vendedor dono da conta (auth_user)	integer	SIM	FK
mp_user_id	ID do usuário no Mercado Pago	varchar(50)	SIM	
access_token	*Token* de acesso usado nas cobranças em nome do vendedor	varchar(255)	SIM	
refresh_token	*Token* usado para renovar o *token* de acesso	varchar(255)	SIM	
public_key	Chave pública da conta no Mercado Pago	varchar(255)	SIM	
connected_at	Data da conexão da conta	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 46 – Dicionário de dados: Avaliações de vendedores (accounts_sellerreview)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da avaliação	integer	SIM	PK

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
seller_id	Vendedor avaliado (auth_user)	integer	SIM	FK
reviewer_id	Usuário que fez a avaliação (auth_ user)	integer	SIM	FK
rating	Nota dada ao vendedor	decimal	SIM	
comment	Comentário da avaliação	text	SIM	
created_at	Data da avaliação	datetime	SIM	
edited	A avaliação já foi editada?	bool	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 47 – Dicionário de dados: Categorias (catalog_category)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da categoria	integer	SIM	PK
name	Nome da categoria	varchar(100)	SIM	
slug	Identificador da categoria usado na URL	varchar(50)	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 48 – Dicionário de dados: Produtos (catalog_product)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do produto	integer	SIM	PK
category_id	Categoria do produto (catalog_ category)	bigint	SIM	FK
seller_id	Vendedor que publicou o anúncio (auth_user)	integer	SIM	FK
title	Título do anúncio	varchar(255)	SIM	
description	Descrição do produto	text	SIM	

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
condition	Condição do produto (novo ou usado)	varchar(4)	SIM	
created_at	Data de criação do anúncio	datetime	SIM	
updated_at	Data da última alteração do anúncio	datetime	SIM	
accepts_pickup	O vendedor aceita a retirada em mãos?	bool	SIM	
accepts_trade	O anúncio aceita propostas de troca?	bool	SIM	
trade_ preferences	O que o vendedor aceita receber em troca	varchar(300)	SIM	
published	O anúncio está publicado (ou é um rascunho)?	bool	SIM	
deleted	O anúncio foi excluído pelo vendedor?	bool	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 49 – Dicionário de dados: Variações de produtos (catalog_productvariant)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da variação	integer	SIM	PK
product_id	Produto ao qual a variação pertence (catalog_product)	bigint	SIM	FK
name	Nome da variação	varchar(100)	SIM	
price	Preço da variação, em reais	decimal	SIM	
quantity	Quantidade em estoque	integer unsigned	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 50 – Dicionário de dados: Imagens de produtos (catalog_productimage)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da imagem	integer	SIM	PK
product_id	Produto ao qual a imagem pertence (catalog_product)	bigint	SIM	FK
image	Caminho do arquivo da imagem	varchar(100)	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 51 – Dicionário de dados: Avaliações de produtos (catalog_productreview)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da avaliação	integer	SIM	PK
product_id	Produto avaliado (catalog_product)	bigint	SIM	FK
reviewer_id	Usuário que fez a avaliação (auth_ user)	integer	SIM	FK
rating	Nota dada ao produto	decimal	SIM	
comment	Comentário da avaliação	text	SIM	
created_at	Data da avaliação	datetime	SIM	
edited	A avaliação já foi editada?	bool	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 52 – Dicionário de dados: Carrinhos (orders_cart)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do carrinho	integer	SIM	PK
user_id	Usuário dono do carrinho (auth_user)	integer	SIM	FK
created_at	Data de criação do carrinho	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 53 – Dicionário de dados: Itens do carrinho (orders_cartitem)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do item	integer	SIM	PK
cart_id	Carrinho ao qual o item pertence (orders_cart)	bigint	SIM	FK
product_id	Produto adicionado (catalog_product)	bigint	SIM	FK
variant_id	Variação escolhida (catalog_ productvariant)	bigint	SIM	FK
quantity	Quantidade do item	integer unsigned	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 54 – Dicionário de dados: Pedidos (orders_order)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do pedido	integer	SIM	PK
buyer_id	Comprador (auth_ user)	integer	SIM	FK
product_id	Produto comprado (catalog_product)	bigint	SIM	FK
variant_id	Variação comprada (catalog_ productvariant)	bigint	SIM	FK
quantity	Quantidade comprada	integer unsigned	SIM	
total_price	Preço total, antes dos descontos	decimal	SIM	
status	Etapa do pedido (aguardando pagamento, pago, enviado, entregue, em disputa, concluído etc.)	varchar(20)	SIM	
pickup	O pedido será retirado em mãos?	bool	SIM	

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
tracking_code	Código de rastreio do envio	varchar(13)	SIM	
coins_used	MegaCoins usadas no pedido	integer unsigned	SIM	
coins_discount	Desconto obtido com as MegaCoins, em reais	decimal	SIM	
seller_coupon_ id	Uso de cupom da loja aplicado ao pedido (coupons_ couponredemption)	bigint	NÃO	FK
seller_coupon_ discount	Desconto do cupom da loja	decimal	SIM	
platform_ coupon_id	Uso de cupom da plataforma aplicado ao pedido (coupons_ couponredemption)	bigint	NÃO	FK
platform_ coupon_ discount	Desconto do cupom da plataforma	decimal	SIM	
trade_id	Troca que originou o pedido, se houver (trades_ tradeproposal)	bigint	NÃO	FK
trade_fee	Parte da taxa de troca cobrada neste pedido	decimal	SIM	
trade_paid	A parte da troca referente a este pedido já foi paga?	bool	SIM	
created_at	Data de criação do pedido	datetime	SIM	
updated_at	Data da última alteração do pedido	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 55 – Dicionário de dados: Comissões (orders_commission)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da comissão	integer	SIM	PK
order_id	Pedido que gerou a comissão (orders_ order)	bigint	SIM	FK
rate	Taxa de comissão aplicada, em porcentagem	decimal	SIM	
gross_amount	Valor bruto da venda	decimal	SIM	
commission_ amount	Valor da comissão retida pela plataforma	decimal	SIM	
net_amount	Valor líquido repassado ao vendedor	decimal	SIM	
created_at	Data de registro da comissão	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 56 – Dicionário de dados: Solicitações de devolução (orders_returnrequest)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da solicitação	integer	SIM	PK
order_id	Pedido cuja devolução foi solicitada (orders_ order)	bigint	SIM	FK
reason_category	Motivo da devolução, escolhido de uma lista	varchar(100)	SIM	
description	Descrição do problema feita pelo comprador	text	SIM	
created_at	Data da solicitação	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 57 – Dicionário de dados: Imagens de solicitações de devolução (orders_returnrequestimage)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da imagem	integer	SIM	PK
return_request_id	Solicitação à qual a imagem pertence (orders_returnrequest)	bigint	SIM	FK
image	Caminho do arquivo da imagem	varchar(100)	SIM	
created_at	Data de envio da imagem	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 58 – Dicionário de dados: Disputas (orders_dispute)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da disputa	integer	SIM	PK
order_id	Pedido em disputa (orders_order)	bigint	SIM	FK
opened_by_id	Usuário que abriu a disputa (auth_user)	integer	SIM	FK
reason_category	Motivo da disputa, escolhido de uma lista	varchar(100)	SIM	
reason	Descrição do motivo	text	SIM	
status	Situação da disputa (aberta, a favor do comprador com ou sem devolução, ou a favor do vendedor)	varchar(25)	SIM	
resolution_notes	Justificativa da decisão do gestor	text	SIM	
resolved_by_id	Gestor que resolveu a disputa (auth_user)	integer	NÃO	FK
created_at	Data de abertura	datetime	SIM	
resolved_at	Data da resolução	datetime	NÃO	

Fonte: elaborado pelo próprio autor.

Quadro 59 – Dicionário de dados: Mensagens das disputas (orders_disputemessage)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da mensagem	integer	SIM	PK
dispute_id	Disputa à qual a mensagem pertence (orders_dispute)	bigint	SIM	FK
author_id	Autor da mensagem (auth_user)	integer	SIM	FK
message	Texto da mensagem	text	SIM	
created_at	Data de envio	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 60 – Dicionário de dados: Evidências de disputa (orders_disputeevidence)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da evidência	integer	SIM	PK
dispute_id	Disputa à qual a evidência pertence (orders_dispute)	bigint	SIM	FK
message_id	Mensagem que acompanha a imagem, se houver (orders_ disputemessage)	bigint	NÃO	FK
uploaded_by_id	Usuário que enviou a evidência (auth_ user)	integer	SIM	FK
image	Caminho do arquivo da imagem	varchar(100)	SIM	
created_at	Data de envio	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 61 – Dicionário de dados: Leilões (auctions_auction)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do leilão	integer	SIM	PK
product_id	Produto leiloado (catalog_product)	bigint	SIM	FK
start_price	Valor do lance inicial	decimal	SIM	

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
reserve_price	Preço mínimo para que o item seja vendido	decimal	NÃO	
buy_now_price	Preço de compra imediata	decimal	NÃO	
duration_days	Duração do leilão, em dias	smallint unsigned	SIM	
starts_at	Data de início	datetime	SIM	
ends_at	Data de término	datetime	SIM	
status	Situação do leilão (em andamento, vendido ou encerrado sem venda)	varchar(6)	SIM	
current_price	Valor do lance atual	decimal	SIM	
bid_count	Quantidade de lances recebidos	integer unsigned	SIM	
leader_id	Participante com o maior lance (auth_ user)	integer	NÃO	FK
winner_id	Vencedor do leilão (auth_user)	integer	NÃO	FK
order_id	Pedido gerado pela venda (orders_ order)	bigint	NÃO	FK
bought_now	O item foi vendido pela compra imediata?	bool	SIM	
ended_early	O vendedor encerrou o leilão antes do prazo?	bool	SIM	
closed_at	Data de encerramento	datetime	NÃO	
payment_due_at	Prazo para o vencedor pagar	datetime	NÃO	

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
relisted_as_id	Novo leilão criado quando o item é relistado (auctions_ auction)	bigint	NÃO	FK

Fonte: elaborado pelo próprio autor.

Quadro 62 – Dicionário de dados: Lances (auctions_bid)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do lance	integer	SIM	PK
auction_id	Leilão do lance (auctions_auction)	bigint	SIM	FK
bidder_id	Participante que deu o lance (auth_user)	integer	SIM	FK
amount	Valor do lance	decimal	SIM	
is_auto	O lance foi dado automaticamente pelo sistema, até o valor máximo do participante?	bool	SIM	
created_at	Data do lance	datetime	SIM	
retracted_at	Data em que o lance foi retirado	datetime	NÃO	
retraction_ reason	Motivo da retirada do lance	varchar(11)	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 63 – Dicionário de dados: Leilões acompanhados (auctions_auctionwatch)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do acompanhamento	integer	SIM	PK
user_id	Usuário que acompanha o leilão (auth_user)	integer	SIM	FK
auction_id	Leilão acompanhado (auctions_auction)	bigint	SIM	FK

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
created_at	Data em que o acompanhamento começou	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 64 – Dicionário de dados: Ofertas de segunda chance (auctions_secondchanceoffer)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da oferta	integer	SIM	PK
auction_id	Leilão de origem (auctions_auction)	bigint	SIM	FK
bidder_id	Participante que recebeu a oferta (auth_user)	integer	SIM	FK
price	Preço oferecido	decimal	SIM	
duration_days	Validade da oferta, em dias	smallint unsigned	SIM	
status	Situação da oferta (aguardando resposta, aceita, recusada ou expirada)	varchar(8)	SIM	
created_at	Data de envio	datetime	SIM	
expires_at	Data de expiração	datetime	SIM	
responded_at	Data da resposta	datetime	NÃO	
order_id	Pedido gerado quando a oferta é aceita (orders_order)	bigint	NÃO	FK

Fonte: elaborado pelo próprio autor.

Quadro 65 – Dicionário de dados: Propostas de troca (trades_tradeproposal)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da proposta	integer	SIM	PK
product_id	Anúncio que recebeu a proposta (catalog_product)	bigint	SIM	FK

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
variant_id	Variação desejada pelo proponente (catalog_ productvariant)	bigint	SIM	FK
proposer_id	Usuário que fez a proposta (auth_user)	integer	SIM	FK
owner_id	Dono do anúncio (auth_user)	integer	SIM	FK
cash_amount	Valor da volta em dinheiro	decimal	SIM	
cash_payer	Lado que paga a volta (proponente ou anunciante)	varchar(8)	SIM	
status	Situação da proposta (em negociação, aceita, recusada, retirada, expirada ou cancelada após o aceite)	varchar(9)	SIM	
awaiting	Lado que deve responder agora	varchar(8)	SIM	
rounds	Número de rodadas de negociação	integer unsigned	SIM	
expires_at	Prazo para a próxima resposta	datetime	SIM	
created_at	Data de criação	datetime	SIM	
updated_at	Data da última alteração	datetime	SIM	
closed_at	Data de encerramento	datetime	NÃO	

Fonte: elaborado pelo próprio autor.

Quadro 66 – Dicionário de dados: Itens oferecidos em trocas (trades_tradeitem)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do item	integer	SIM	PK

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
proposal_id	Proposta à qual o item pertence (trades_ tradeproposal)	bigint	SIM	FK
product_id	Produto oferecido pelo proponente (catalog_product)	bigint	SIM	FK
variant_id	Variação oferecida (catalog_ productvariant)	bigint	SIM	FK

Fonte: elaborado pelo próprio autor.

Quadro 67 – Dicionário de dados: Eventos de troca (trades_tradeevent)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do evento	integer	SIM	PK
proposal_id	Proposta à qual o evento pertence (trades_ tradeproposal)	bigint	SIM	FK
author_id	Usuário que realizou a ação (auth_user)	integer	NÃO	FK
kind	Ação registrada (proposta, contraproposta, aceite, recusa, retirada, expiração ou cancelamento)	varchar(9)	SIM	
summary	Termos da proposta naquele momento	text	SIM	
message	Mensagem enviada junto com a ação	text	SIM	
created_at	Data do evento	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 68 – Dicionário de dados: Cupons (coupons_coupon)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do cupom	integer	SIM	PK
code	Código digitado pelo comprador	varchar(20)	SIM	
seller_id	Loja dona do cupom (vazio nos cupons da plataforma) (auth_user)	integer	NÃO	FK
kind	Tipo de desconto (percentual ou valor fixo)	varchar(7)	SIM	
value	Valor do desconto	decimal	SIM	
max_discount	Desconto máximo, em reais	decimal	NÃO	
min_order_value	Valor mínimo da compra, em reais	decimal	SIM	
category_id	Categoria à qual o cupom se restringe, se houver (catalog_ category)	bigint	NÃO	FK
first_purchase_ only	O cupom vale só para a primeira compra?	bool	SIM	
is_public	O cupom aparece na lista de cupons disponíveis?	bool	SIM	
starts_at	Início da validade	datetime	SIM	
ends_at	Fim da validade	datetime	NÃO	
usage_limit	Limite total de usos	integer unsigned	NÃO	
per_user_limit	Limite de usos por cliente	integer unsigned	SIM	
active	O cupom está ativo?	bool	SIM	
created_at	Data de criação	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 69 – Dicionário de dados: Usos de cupons (coupons_couponredemption)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do uso	integer	SIM	PK
coupon_id	Cupom utilizado (coupons_coupon)	bigint	SIM	FK
user_id	Usuário que usou o cupom (auth_user)	integer	SIM	FK
discount	Desconto concedido, em reais	decimal	SIM	
created_at	Data do uso	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 70 – Dicionário de dados: Cupons salvos (coupons_savedcoupon)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID do registro	integer	SIM	PK
user_id	Usuário que salvou o cupom (auth_user)	integer	SIM	FK
coupon_id	Cupom salvo (coupons_coupon)	bigint	SIM	FK
created_at	Data em que o cupom foi salvo	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 71 – Dicionário de dados: Movimentações de MegaCoins (rewards_cointransaction)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da movimentação	integer	SIM	PK
user_id	Usuário dono das moedas (auth_user)	integer	SIM	FK
kind	Tipo de movimentação (*cashback*, avaliação, *check-in*, desconto em compra, devolução ou ajuste manual)	varchar(10)	SIM	

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
amount	Quantidade de moedas (negativa nos débitos)	integer	SIM	
order_id	Pedido relacionado, se houver (orders_ order)	bigint	NÃO	FK
reference	Identificador que impede créditos repetidos para o mesmo evento	varchar(50)	SIM	
description	Descrição exibida no extrato	varchar(200)	SIM	
created_at	Data da movimentação	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 72 – Dicionário de dados: Despesas (orders_expense)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da despesa	integer	SIM	PK
description	Descrição da despesa	varchar(120)	SIM	
category	Categoria da despesa (infraestrutura, assinaturas, *marketing*, taxas, pessoal, impostos ou outros)	varchar(20)	SIM	
amount	Valor, em reais	decimal	SIM	
date	Data da despesa ou, nas mensais, da primeira cobrança	date	SIM	
recurring	A despesa se repete todo mês?	bool	SIM	
end_date	Data da última cobrança das despesas mensais	date	NÃO	

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
created_by_id	Gestor que lançou a despesa (auth_user)	integer	NÃO	FK
created_at	Data do lançamento	datetime	SIM	

Fonte: elaborado pelo próprio autor.

Quadro 73 – Dicionário de dados: Configurações da plataforma (orders_ platformconfig)

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
id	ID da configuração (registro único)	integer	SIM	PK
commission_ rate	Taxa de comissão sobre as vendas, em porcentagem	decimal	SIM	
dispute_ window_days	Prazo para contestar um cancelamento sem devolução, em dias	integer unsigned	SIM	
return_window_ days	Prazo para solicitar devolução após a entrega, em dias	integer unsigned	SIM	
seller_ response_days	Prazo de resposta do vendedor antes que o comprador possa abrir disputa, em dias	integer unsigned	SIM	
escalation_ window_days	Prazo para o comprador abrir disputa depois do prazo do vendedor, em dias	integer unsigned	SIM	
ranking_size	Quantidade de vendedores no ranking do *dashboard*	integer unsigned	SIM	
dispute_reasons	Motivos de disputa disponíveis, um por linha	text	SIM	

Nome	Conteúdo	Tipo	Obrigatório?	Chave?
return_reasons	Motivos de devolução disponíveis, um por linha	text	SIM	
shelf_size	Quantidade de produtos por prateleira da página inicial	integer unsigned	SIM	
coins_ cashback_rate	*Cashback* em MegaCoins, em porcentagem do valor pago	decimal	SIM	
coins_max_ redeem_rate	Desconto máximo com MegaCoins por pedido, em porcentagem	decimal	SIM	
coins_review_ reward	MegaCoins concedidas por avaliação	integer unsigned	SIM	
auction_ payment_days	Prazo para o vencedor de um leilão pagar, em dias	integer unsigned	SIM	
second_chance_ window_days	Prazo para enviar ofertas de segunda chance, em dias	integer unsigned	SIM	
trade_response_ days	Prazo para responder uma proposta de troca, em dias	integer unsigned	SIM	
trade_fee	Taxa fixa cobrada por troca, em reais	decimal	SIM	

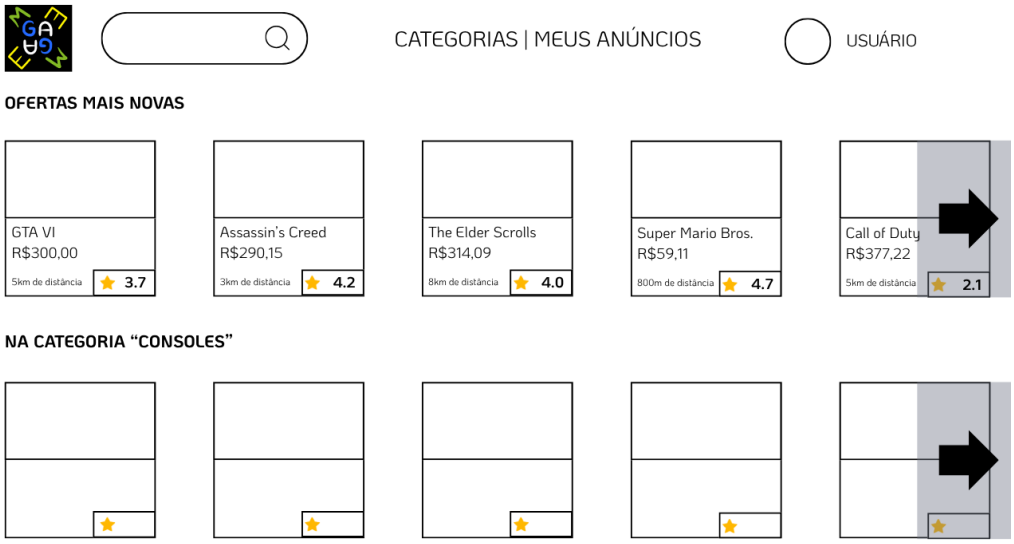
Fonte: elaborado pelo próprio autor.

3.7 Protótipos de baixo nível

Os protótipos de baixo nível (*wireframes*) foram elaborados antes da implementação para definir a estrutura das principais páginas do MegaGame. A Figura 17 apre-

senta a página inicial; as Figuras 18 e 19, as páginas de categorias e de anúncios; a Figura 20, o cadastro; a Figura 21, o menu do usuário; e a Figura 22, o manual de usuário.

Figura 17 – *Wireframe* da página inicial do MegaGame



Fonte: elaborado pelo próprio autor.

Figura 18 – *Wireframe* da página de categorias do MegaGame



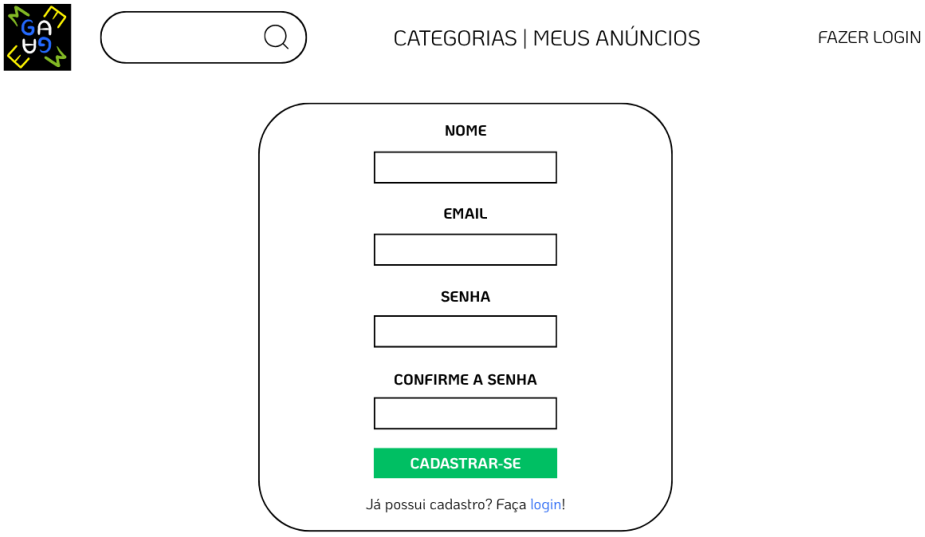
Fonte: elaborado pelo próprio autor.

Figura 19 – *Wireframe* da página de anúncios do MegaGame



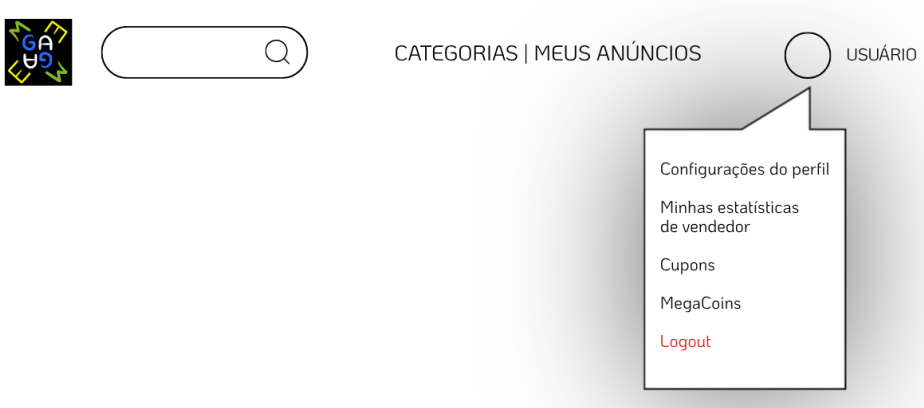
Fonte: elaborado pelo próprio autor.

Figura 20 – *Wireframe* da página de cadastro do MegaGame



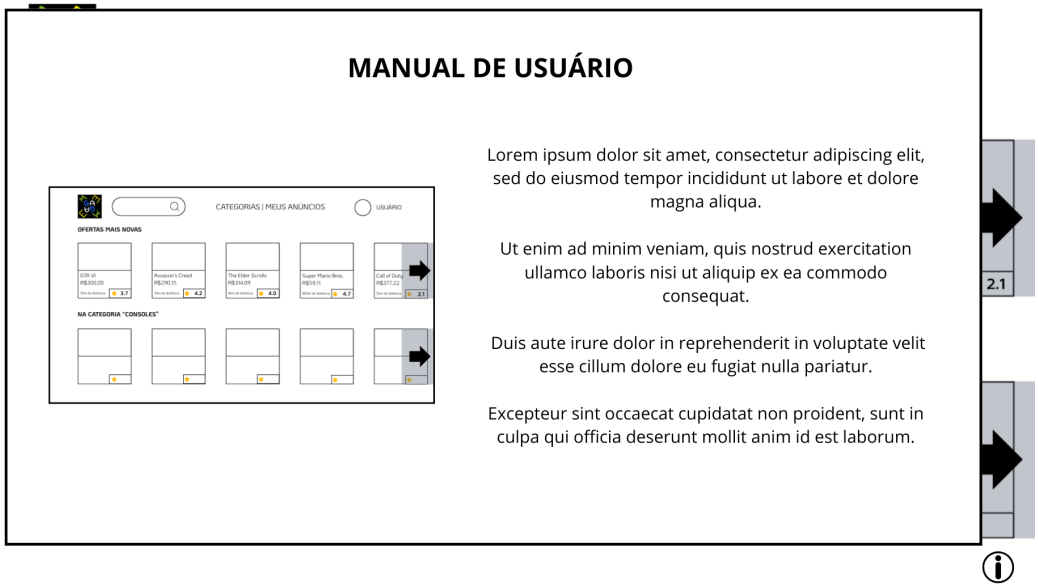
Fonte: elaborado pelo próprio autor.

Figura 21 – *Wireframe* do menu do usuário do MegaGame



Fonte: elaborado pelo próprio autor.

Figura 22 – *Wireframe* do manual de usuário do MegaGame



Fonte: elaborado pelo próprio autor.

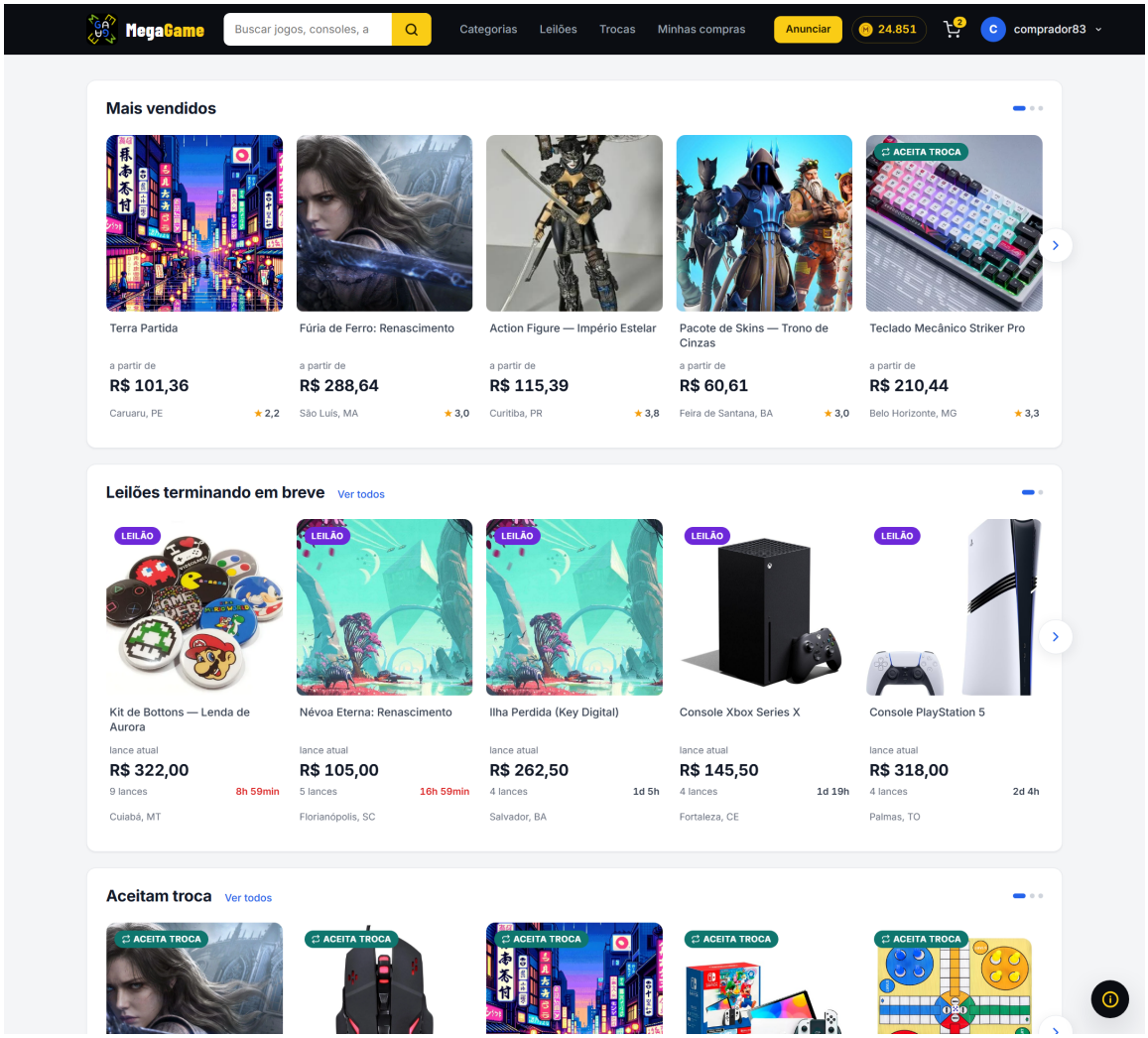
3.8 Interfaces

Esta seção apresenta uma seleção das telas do MegaGame, e não todas elas. Foram escolhidas as que concentram as principais decisões de *design* visual e as funcionalidades centrais da plataforma; o uso de cada tela é descrito no Manual de usuário (Seção 3.11). As capturas foram feitas com os dados de exemplo usados nos testes, e não com dados de usuários reais.

A identidade visual é a mesma em todas as páginas. As cores ficam definidas em variáveis CSS, numa única folha de estilos, o que mantém cada tom igual em qualquer tela: azul para as ações principais, amarelo como cor de destaque da marca e verde, laranja e vermelho para sucesso, aviso e erro. Duas cores são reservadas a funcionalidades específicas: o violeta, usado nos leilões e nos cupons, e o verde-azulado, usado nas trocas. O texto usa a fonte Inter, escolhida pela boa leitura em tamanhos pequenos, e o nome da marca usa a Barlow Condensed, mais estreita e pesada. O conteúdo fica em cartões brancos, de cantos arredondados e sombra leve, sobre um fundo cinza-claro, e o cabeçalho escuro permanece fixo no topo durante a rolagem.

A página inicial (Figura 23) foi escolhida por ser o primeiro contato do usuário com o site e por reunir quase todos os elementos da identidade visual. O cabeçalho escuro destaca o logotipo e o botão amarelo de anunciar, que convida o usuário a vender, enquanto o saldo de MegaCoins e o carrinho ficam sempre à vista. Os produtos aparecem em prateleiras horizontais, cada uma com um título e um único tipo de conteúdo, para que a página possa ser percorrida rapidamente. Nos cartões, o preço tem o maior peso visual, e as etiquetas coloridas no canto da imagem identificam leilões e anúncios que aceitam troca antes mesmo da leitura do título.

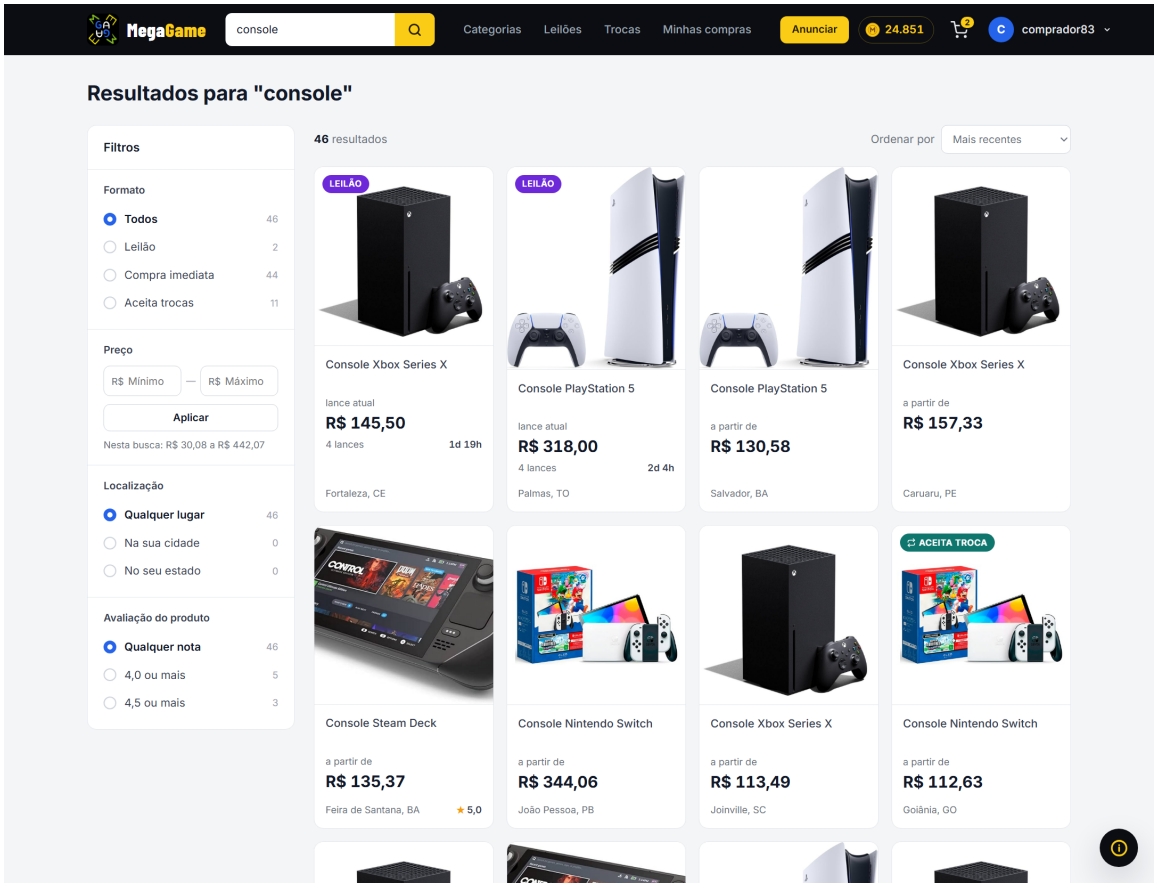
Figura 23 – Captura de tela da página inicial do MegaGame



Fonte: elaborado pelo próprio autor.

A tela de busca (Figura 24) é aquela em que o comprador passa mais tempo ao procurar um produto. Os filtros ficam numa coluna à esquerda, separados em grupos, e cada opção mostra quantos resultados retorna, o que evita filtros que levam a uma lista vazia. Os resultados usam os mesmos cartões da página inicial, em grade de quatro colunas, de modo que o usuário reconhece o padrão em qualquer listagem. Nos leilões, o cartão troca o “a partir de” pelo lance atual e acrescenta a quantidade de lances e o tempo restante, que fica vermelho quando falta menos de um dia.

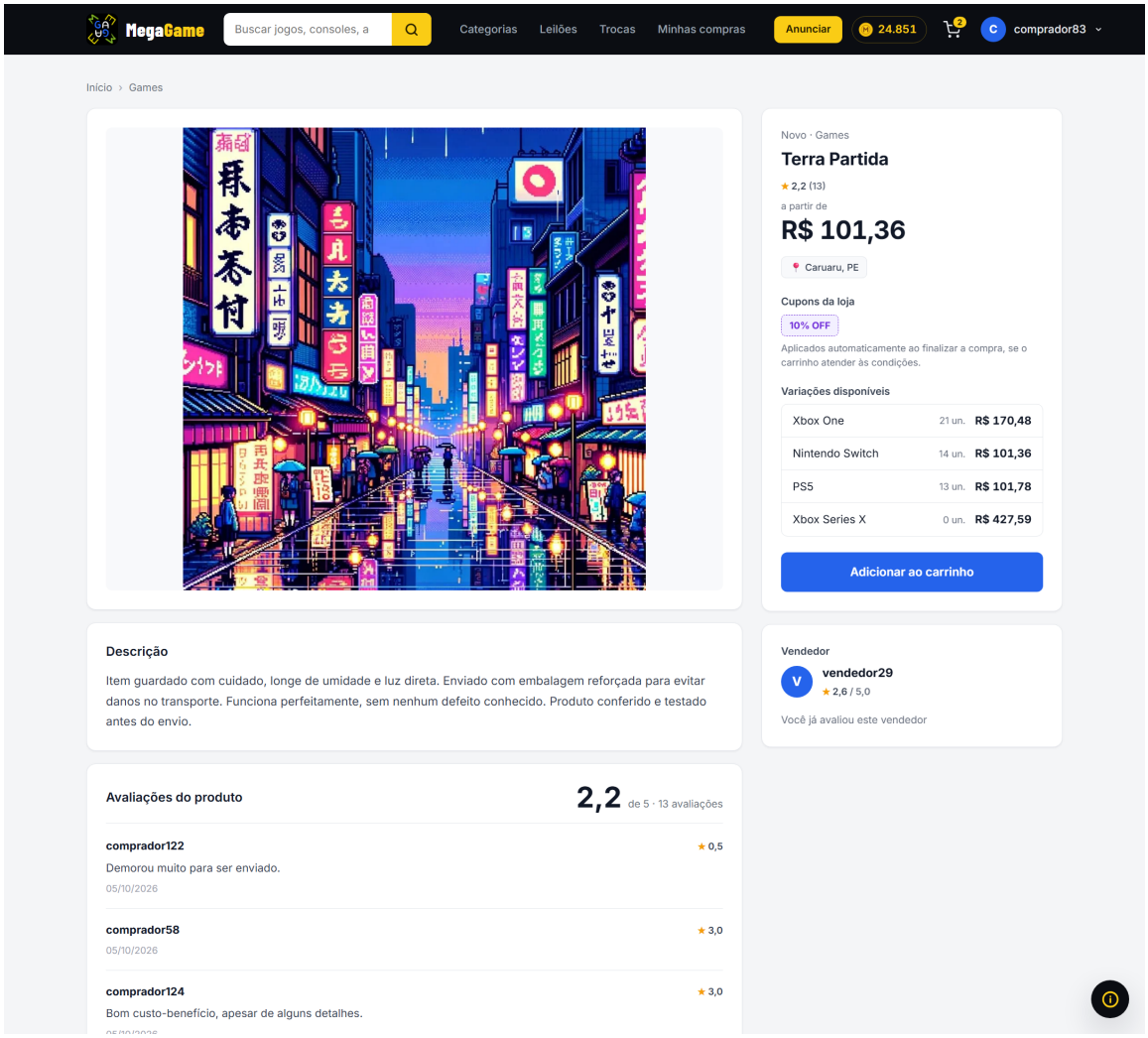
Figura 24 – Captura de tela da busca com filtros do MegaGame



Fonte: elaborado pelo próprio autor.

A página de produto (Figura 25) é onde o comprador decide a compra. O *layout* divide a tela em duas colunas: à esquerda, a imagem em destaque, seguida da descrição e das avaliações; à direita, um painel com o que pesa na decisão, em ordem de importância: título, nota, preço, localização, cupons da loja e variações. Cada variação ocupa uma linha, com estoque e preço alinhados à direita para facilitar a comparação, e o botão azul de adicionar ao carrinho é a única ação primária da tela. Os cupons aparecem em violeta e com borda tracejada, o mesmo visual de cupom usado no restante do site.

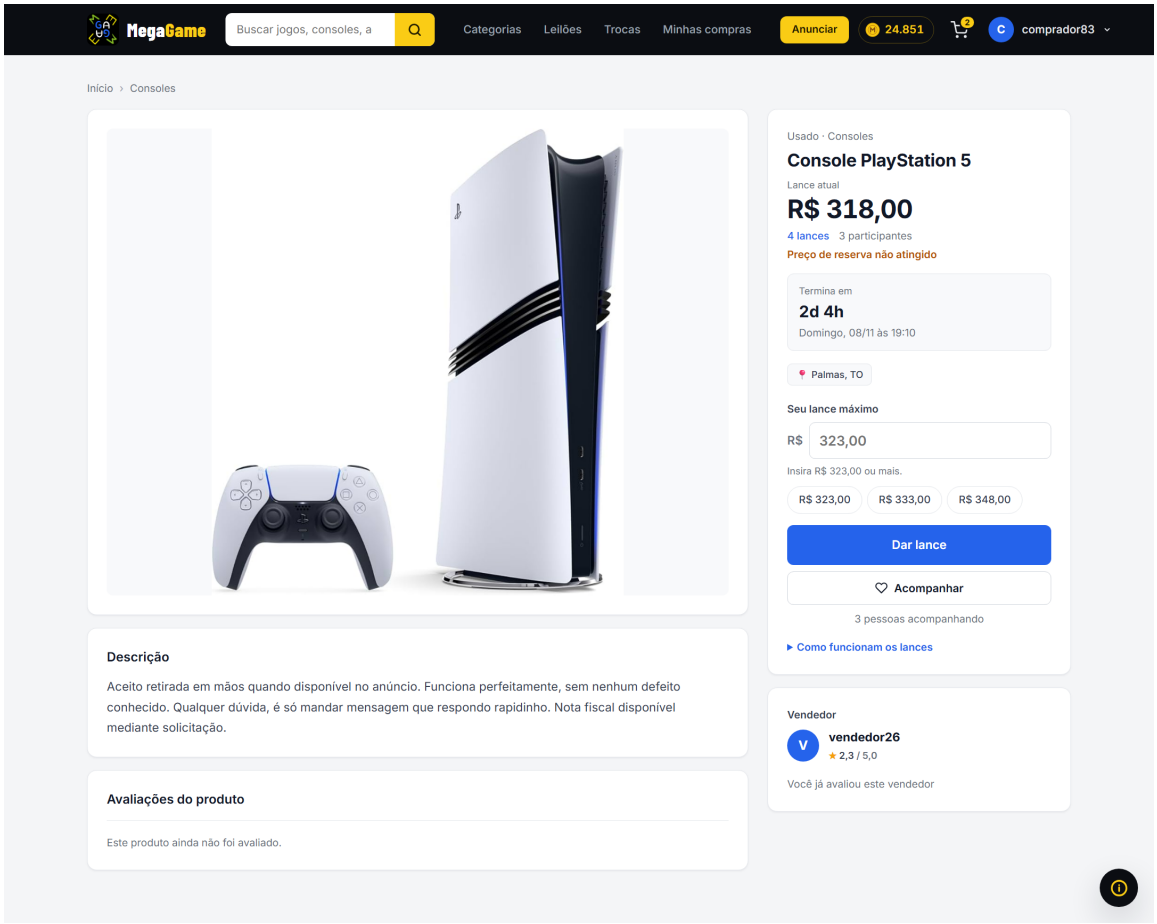
Figura 25 – Captura de tela da página de produto do MegaGame



Fonte: elaborado pelo próprio autor.

A tela de leilão (Figura 26) foi incluída por mostrar como a página de produto se adapta a uma modalidade de venda diferente da compra direta. A estrutura em duas colunas se mantém, mas o painel da direita passa a destacar o lance atual, o tempo restante, num bloco próprio, e o aviso de preço de reserva, em laranja por ser uma informação de alerta. Os valores sugeridos aparecem como botões em formato de pílula logo abaixo do campo de lance, e a opção de acompanhar o leilão fica num botão secundário, só com contorno, para não competir com o botão principal.

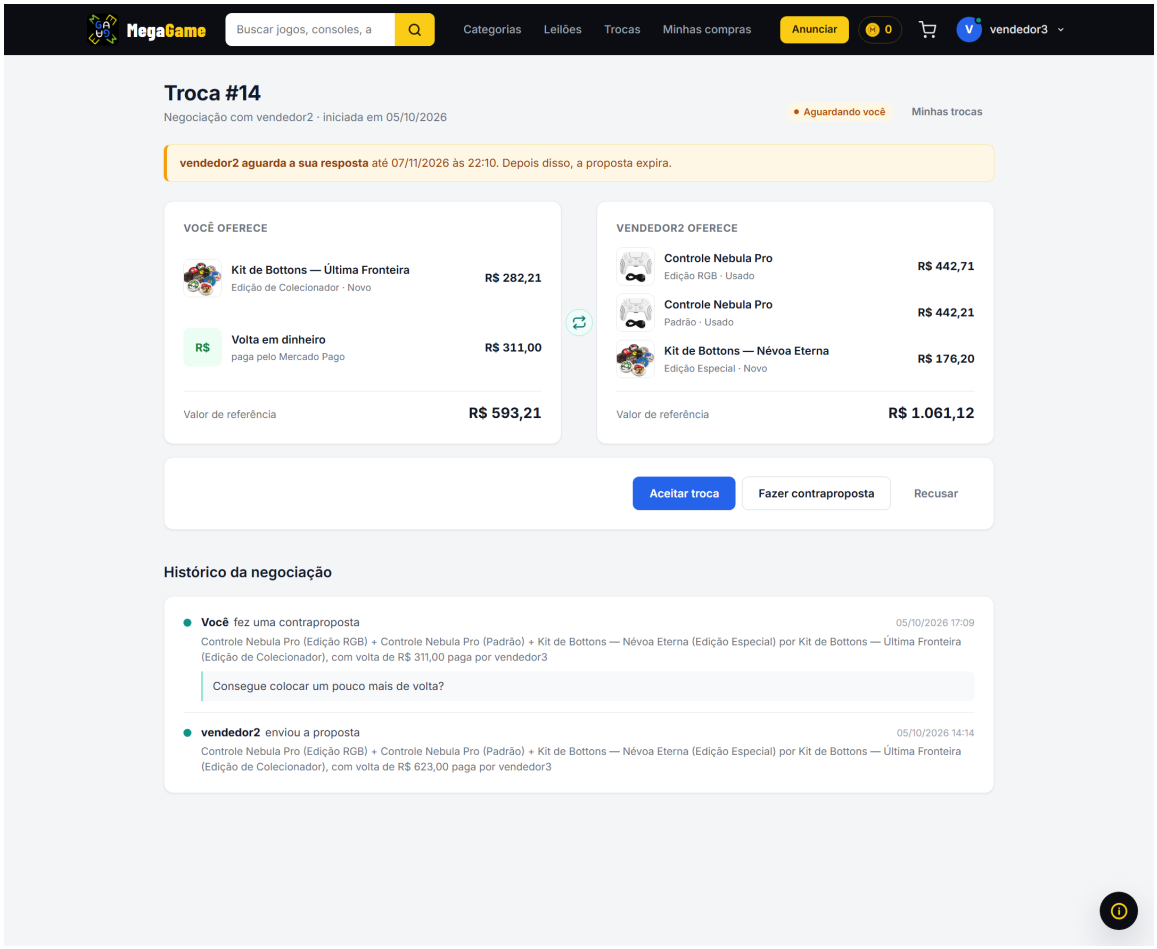
Figura 26 – Captura de tela de um leilão no MegaGame



Fonte: elaborado pelo próprio autor.

A negociação de troca (Figura 27) representa a terceira forma de negociação da plataforma, ao lado da compra direta e do leilão, e é a interface mais diferente do restante do site. As duas ofertas ficam lado a lado, em cartões do mesmo tamanho ligados por um ícone de troca ao centro, para que o usuário compare o que dá com o que recebe. A volta em dinheiro entra na lista como mais um item, e o valor de referência de cada lado fecha os cartões. Uma faixa amarela no topo indica de quem é a vez e até quando a proposta vale, e o histórico da negociação usa marcadores em verde-azulado, a cor das trocas.

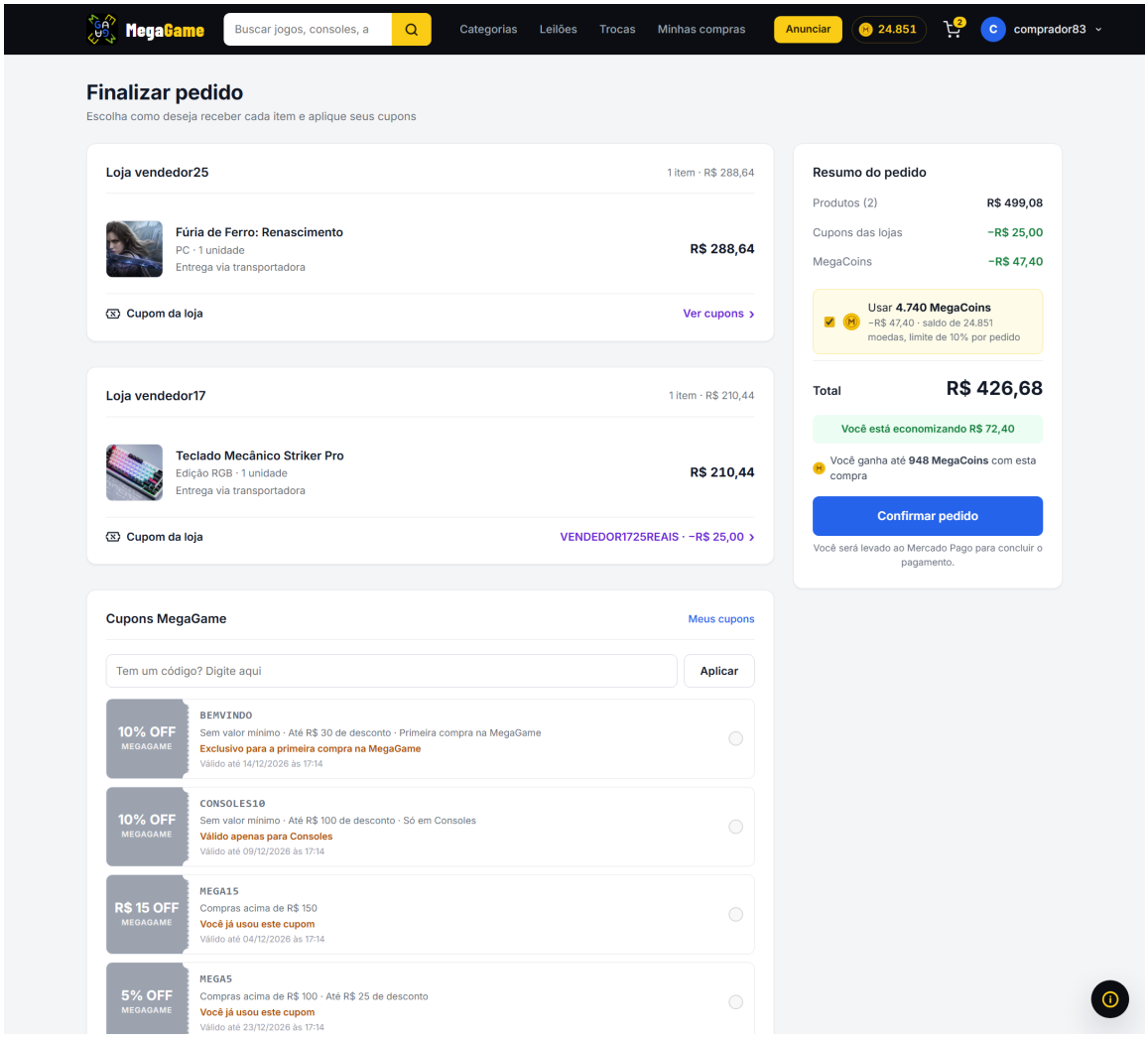
Figura 27 – Captura de tela de uma negociação de troca no MegaGame



Fonte: elaborado pelo próprio autor.

A finalização do pedido (Figura 28) foi escolhida por reunir numa só tela os descontos da plataforma. Os itens são agrupados por loja, já que cada vendedor tem os próprios cupons, e o resumo à direita mostra cada desconto numa linha separada, em verde, antes do total. O uso de MegaCoins fica num quadro amarelo, a cor das moedas, e uma faixa verde informa quanto o comprador está economizando. Os cupons foram desenhados como tíquetes, com o desconto em destaque numa aba lateral e um recorte que imita papel picotado; quando um cupom não pode ser usado, a aba fica cinza e o motivo aparece em laranja.

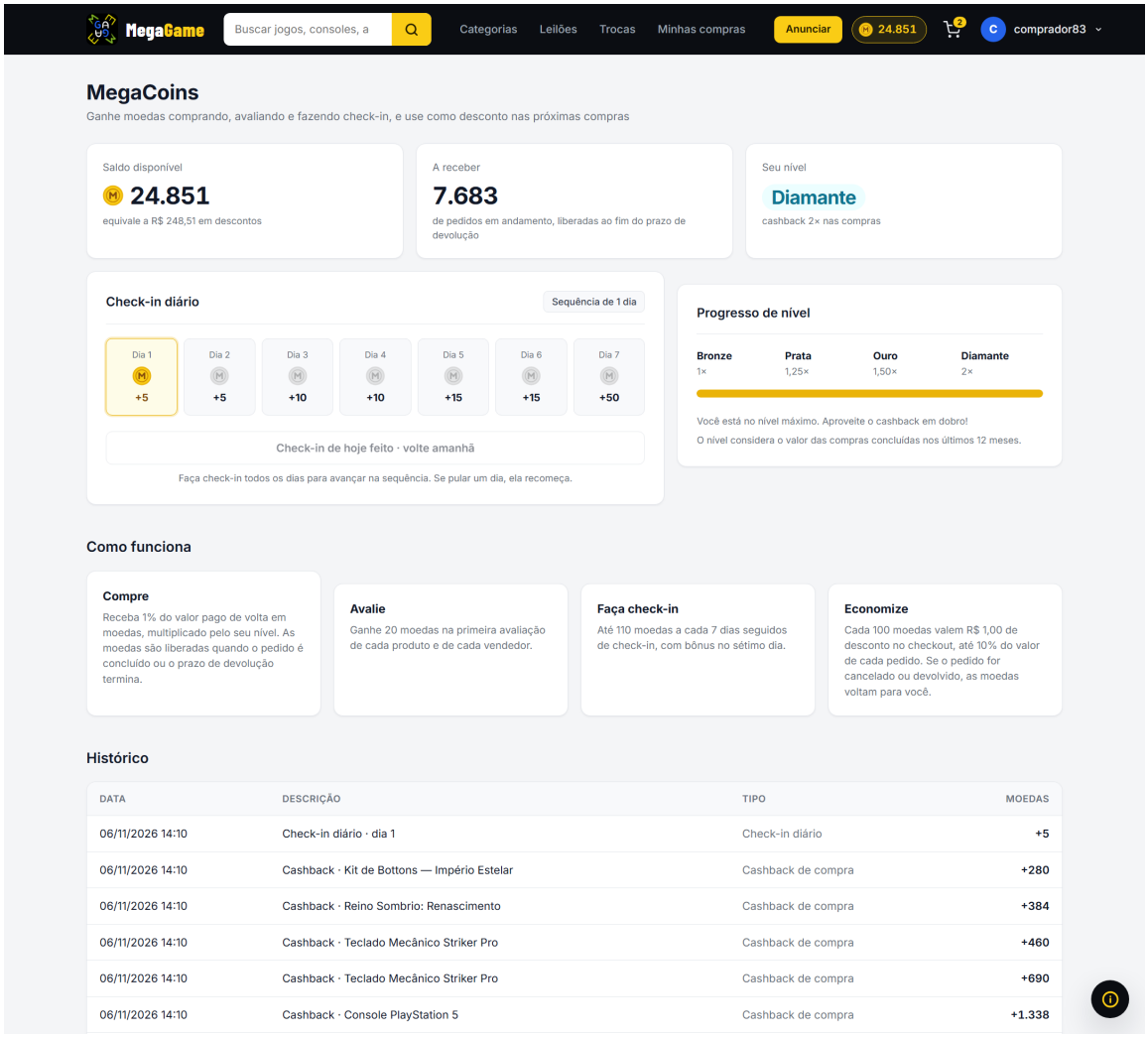
Figura 28 – Captura de tela da finalização do pedido no MegaGame



Fonte: elaborado pelo próprio autor.

A carteira de MegaCoins (Figura 29) mostra a parte de gamificação do sistema, que pedia uma linguagem visual própria. O saldo, as moedas a receber e o nível ficam em três cartões no topo, com números grandes. O *check-in* diário é representado por sete blocos, um para cada dia da sequência, com o dia atual destacado em amarelo, e o progresso de nível usa uma barra com os quatro níveis e seus multiplicadores. Uma seção com quatro cartões curtos explica como as moedas são ganhas e usadas, e o histórico, em forma de tabela, fecha a página.

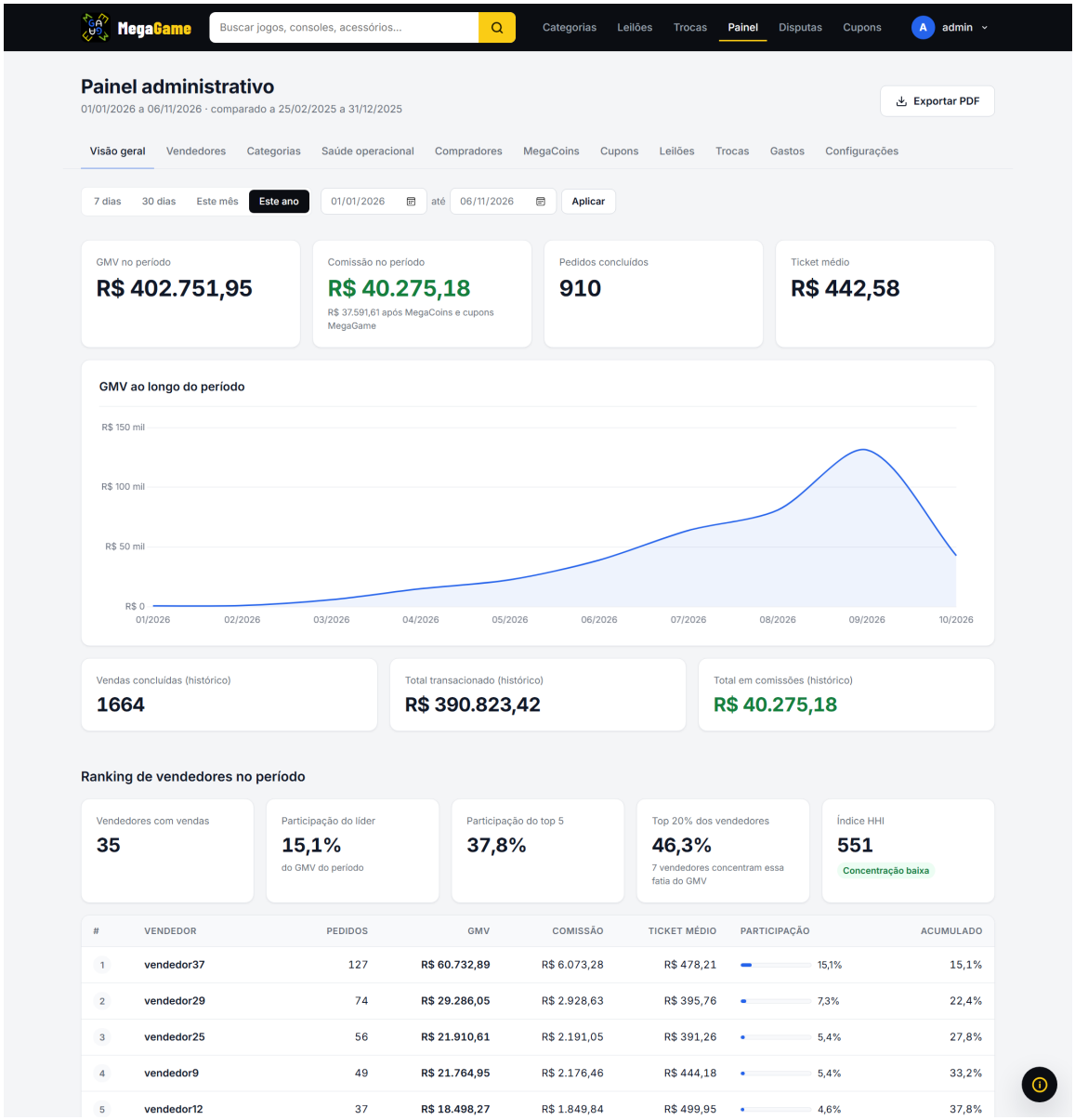
Figura 29 – Captura de tela da carteira de MegaCoins



Fonte: elaborado pelo próprio autor.

Por fim, o *dashboard* do gestor (Figura 30) representa a área administrativa, que tem prioridades diferentes das telas voltadas aos compradores. A página é organizada em abas, para dividir um grande volume de informação, e começa pelos indicadores principais em cartões, com a comissão em verde por ser a receita da plataforma. Em seguida vêm o gráfico do valor bruto das mercadorias vendidas (*gross merchandise value* – GMV) ao longo do período e o *ranking* de vendedores, cujas barras de participação mostram a concentração das vendas sem que seja preciso ler os números.

Figura 30 – Captura de tela do *dashboard* do gestor do MegaGame



Fonte: elaborado pelo próprio autor.

3.9 Implementação

O projeto foi construído no Visual Studio Code, ambiente de desenvolvimento integrado (*integrated development environment* – IDE) da Microsoft. Para executá-lo, é preciso instalar as dependências listadas no arquivo requirements.txt em um ambiente virtual (venv) e criar um arquivo .env a partir do modelo .env.example, com a chave secreta do Django e as credenciais do Mercado Pago. Os campos de conexão com o banco de dados só precisam ser preenchidos para usar o PostgreSQL, caso em que o bloco de configuração correspondente em core/settings.py substitui o do SQLite. Em seguida, o comando “python manage.py migrate” cria as tabelas no banco, e o co-

mando “python manage.py runserver” inicia o servidor local. Para testes, o comando “python manage.py seed_db” preenche o banco com usuários, anúncios, pedidos, leilões e trocas fictícios. Já para que as notificações de pagamento do Mercado Pago cheguem ao servidor local, ele precisa ser exposto pelo túnel do ngrok descrito na Seção 1.3.

O arquivo manage.py, localizado na raiz do repositório, é o utilitário de linha de comando do Django, usado para executar as tarefas administrativas do projeto, como aplicar as *migrations*, gerar os dados de teste e iniciar o servidor.

3.10 Testes

Os testes do MegaGame foram, em sua maioria, de usabilidade. A cada funcionalidade concluída, os fluxos correspondentes eram percorridos no navegador com contas de comprador, de vendedor e de gestor, para verificar se cada etapa era compreensível e se o sistema respondia como esperado, inclusive nas mensagens de erro e nos casos-limite. Os pagamentos foram testados no ambiente de *sandbox* do Mercado Pago, sem transações reais (Seção 1.3). Quando um problema aparecia, ele era corrigido e o fluxo era percorrido de novo.

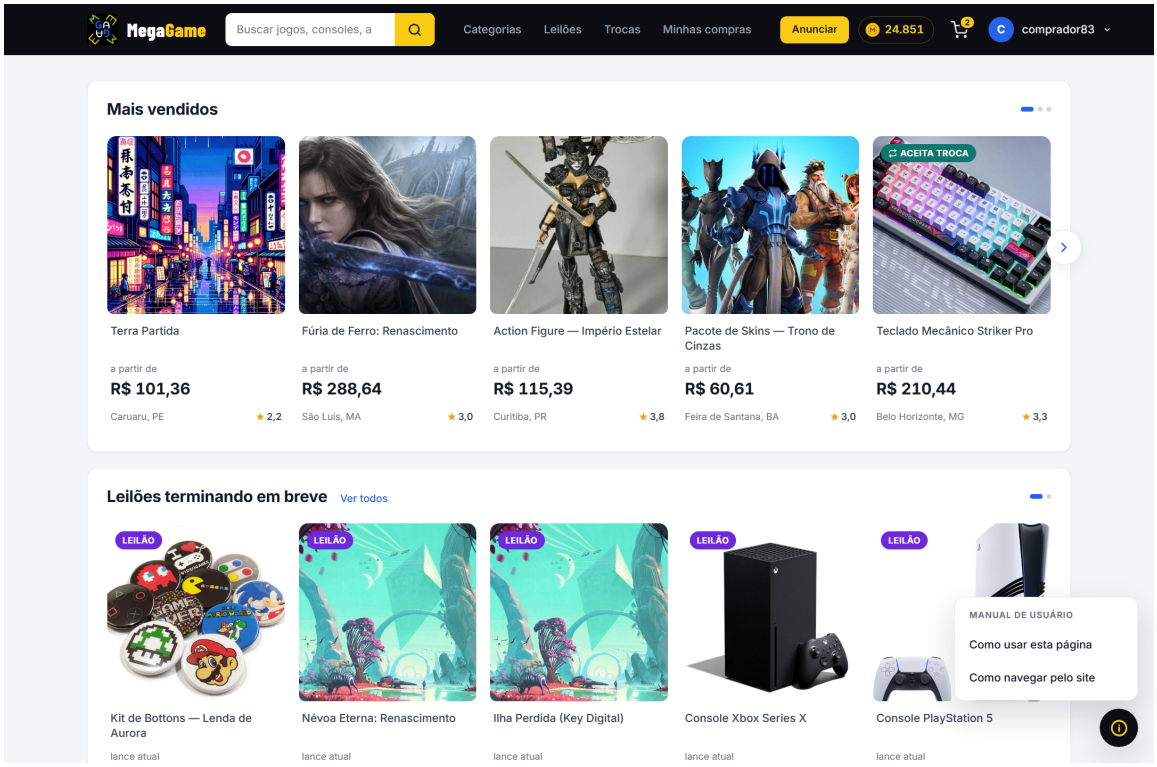
Testes unitários foram usados de forma pontual. Eles foram escritos com as ferramentas de teste do próprio Django, baseadas no módulo *unittest* da biblioteca padrão do Python. Os testes que acessam o banco de dados não usam os dados reais: um banco separado e vazio é criado para a execução e destruído ao final dela (Django Software Foundation, [2026?c]).

3.11 Manual de usuário

O manual de usuário do MegaGame não é um documento à parte: ele fica dentro do próprio sistema, na forma de tutoriais interativos. Assim, a explicação aparece ao lado do elemento a que se refere, e as instruções acompanham as telas sempre que elas mudam.

Todas as páginas têm um botão de ajuda fixo no canto inferior direito. Na primeira visita, uma dica convida o usuário a usá-lo; depois que o manual é aberto uma vez, a dica deixa de aparecer. O botão abre um menu com dois tutoriais (Figura 31): um geral, sobre como navegar pelo site, e outro específico da página em que o usuário está.

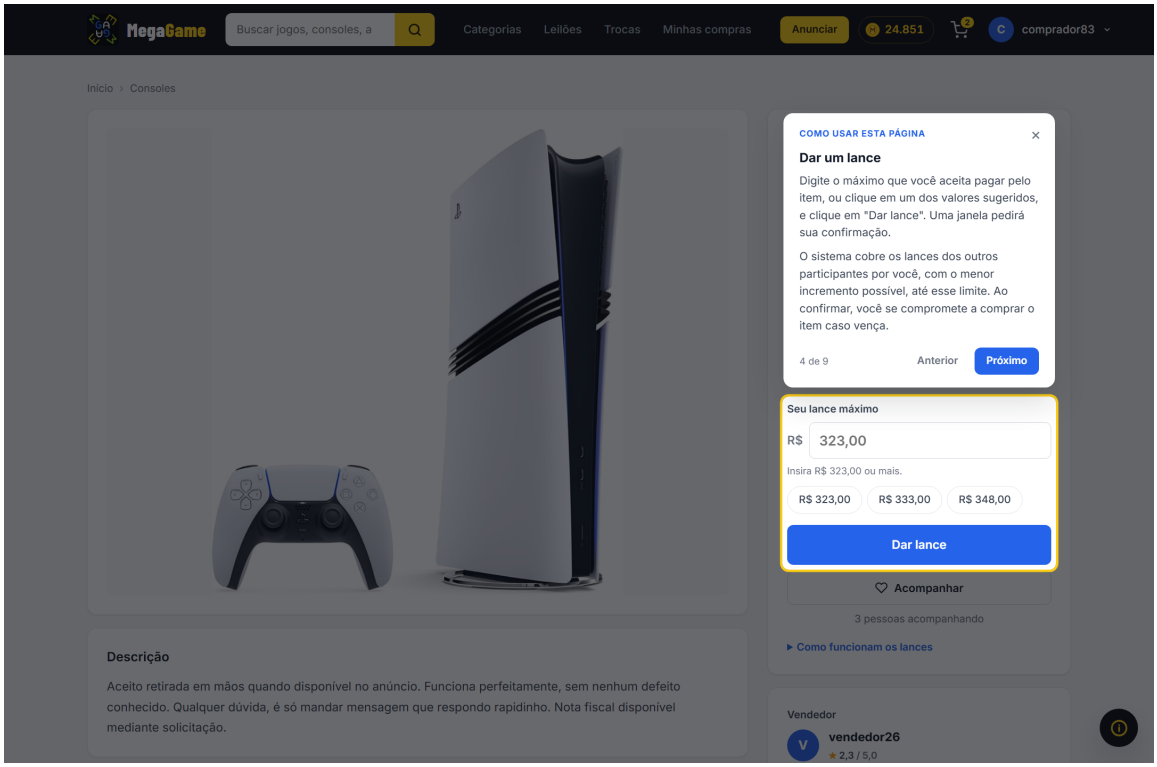
Figura 31 – Captura de tela do menu do manual de usuário



Fonte: elaborado pelo próprio autor.

Ao iniciar um tutorial, a tela escurece e cada passo destaca um elemento da página, acompanhado de uma caixa com título, explicação e a posição no tutorial (Figura 32). O usuário avança e volta pelos botões da caixa ou pelas setas do teclado, e fecha o tutorial a qualquer momento. Os passos mudam conforme quem está usando a página: no leilão, por exemplo, o dono do anúncio vê instruções diferentes das de um comprador, e passos sobre elementos que não estão na tela são pulados. Em telas estreitas, como a de um celular, a caixa fica na parte de baixo, para não cobrir o elemento destacado.

Figura 32 – Captura de tela de um tutorial do manual de usuário



Fonte: elaborado pelo próprio autor.

Na implementação, os passos de cada página são escritos em um *template* HTML próprio, em que cada passo indica o elemento a ser destacado, por meio de um seletor CSS, e o texto a ser exibido. Como os *templates* são processados pelo Django, os passos podem usar as mesmas condições do restante do sistema para se adaptar ao perfil e à situação do usuário. Um *context processor* identifica a rota da página atual e inclui o *template* correspondente, e um *script* em JavaScript, sem bibliotecas externas, monta os tutoriais e posiciona o destaque e a caixa na tela. Ao todo, o manual cobre 38 páginas, com cerca de 200 passos contando as variações por perfil, além dos 11 passos do tutorial geral de navegação.

4 CONCLUSÃO

O objetivo deste trabalho era desenvolver o MegaGame, um *marketplace* C2C dedicado a jogos eletrônicos e produtos relacionados, voltado a um segmento que no Brasil ainda não contava com uma plataforma de comércio própria. O sistema desenvolvido cumpre esse objetivo: compradores e vendedores negociam entre si por compra direta, leilão ou troca, e os pagamentos passam pelo Mercado Pago, que repassa o valor de cada venda diretamente ao vendedor, já descontada a comissão da plataforma.

Além da negociação em si, o projeto cobre o que acontece depois dela e o que incentiva o usuário a voltar. O pós-venda reúne rastreio, cancelamento, devolução e disputas mediadas pelo gestor; os cupons e as MegaCoins oferecem descontos e recompensas; e os *dashboards*, com o relatório em PDF, dão ao gestor e aos vendedores uma visão da parte financeira. A modelagem apresentada neste documento, com requisitos, casos de uso, DERs e dicionários de dados, registra como essas partes se relacionam e deve facilitar a manutenção e a evolução do sistema.

O MegaGame, porém, ainda não está em produção. Conforme a Seção 1.3, os pagamentos funcionam no ambiente de *sandbox* do Mercado Pago, o rastreio dos envios é simulado e o banco de dados é o SQLite. Os próximos passos são hospedar a aplicação, integrar um serviço real de rastreio e acrescentar o cálculo de frete. Cumpridas essas etapas, o sistema poderá ser testado com usuários reais e avaliado frente às metas de desempenho e disponibilidade definidas nos requisitos não funcionais.

REFERÊNCIAS

CORONEL, C.; MORRIS, S. *Database systems*: design, implementation, & management. 14th ed. Boston: Cengage Learning, 2022.

DJANGO SOFTWARE FOUNDATION. *Applications*. [2026?a]. Disponível em: https://docs.djangoproject.com/en/6.0/ref/applications/. Acesso em: 6 nov. 2026.

DJANGO SOFTWARE FOUNDATION. *Official Django logos*. [2026?b]. Disponível em: https://www.djangoproject.com/community/logos/. Acesso em: 5 nov. 2026.

DJANGO SOFTWARE FOUNDATION. *Writing and running tests*. [2026?c]. Disponível em: https://docs.djangoproject.com/en/6.0/topics/testing/overview/. Acesso em: 6 nov. 2026.

DJANGO SOFTWARE FOUNDATION. *Writing your first Django app, part 2*. [2026?d]. Disponível em: https://docs.djangoproject.com/en/6.0/intro/tutorial02/. Acesso em: 6 nov. 2026.

GUEDES, G. T. A. *UML 2*: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018.

NEWZOO. *Top countries and markets by gaming revenues*. 2025. Disponível em: https://newzoo.com/resources/rankings/top-10-countries-by-game-revenues. Acesso em: 22 jun. 2026.

OBJECT MANAGEMENT GROUP. *UML*: Unified Modeling Language. [2026?]. Disponível em: https://www.omg.org/uml/. Acesso em: 5 nov. 2026.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software*: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

PYTHON SOFTWARE FOUNDATION. *The Python logo*. [2026?]. Disponível em: https://www.python.org/community/logos/. Acesso em: 5 nov. 2026.

SOMMERVILLE, I. *Engenharia de software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SQLITE. *About SQLite*. [2026?a]. Disponível em: https://www.sqlite.org/about.html. Acesso em: 6 nov. 2026.

SQLITE. *Appropriate uses for SQLite*. [2026?b]. Disponível em: https://www.sqlite.org/whentouse.html. Acesso em: 6 nov. 2026.

SQLITE. *Most widely deployed SQL database engine*. [2026?c]. Disponível em: https://www.sqlite.org/mostdeployed.html. Acesso em: 6 nov. 2026.

SQLITE. *SQLite developers*. [2026?d]. Disponível em: https://www.sqlite.org/crew.html. Acesso em: 6 nov. 2026.

SUPABASE. *Brand guidelines*: logos and icons. [2026?]. Disponível em: https://supabase.com/brand-assets. Acesso em: 5 nov. 2026.

TANTAU, T. *The TikZ and PGF packages*: manual for version 3.1.12. [*S. l.*]: CTAN, 2026. Disponível em: https://ctan.org/pkg/pgf. Acesso em: 6 nov. 2026.

WALTON, P.; POLLARD, B. *Largest Contentful Paint (LCP)*. [*S. l.*]: web.dev, 2025. Disponível em: https://web.dev/articles/lcp. Acesso em: 6 nov. 2026.

GLOSSÁRIO

Este glossário reúne, em ordem alfabética, os termos técnicos e os estrangeirismos empregados ao longo do trabalho.

App No Django, pacote Python que reúne um conjunto de funcionalidades do sistema, com seus próprios modelos, *views* e rotas.

Back end Parte do sistema executada no servidor, responsável pelas regras de negócio e pelo acesso ao banco de dados.

Brainstorming Técnica de geração de ideias em grupo, em que as propostas são reunidas livremente antes de serem avaliadas.

Cashback Devolução de parte do valor pago numa compra; no MegaGame, é feita em MegaCoins.

Check-in Registro diário de presença do usuário na plataforma, recompensado com MegaCoins.

Context processor No Django, função que acrescenta variáveis a todos os *templates* renderizados, sem que cada página precise fornecê-las.

Dashboard Painel que reúne indicadores e gráficos para acompanhamento rápido de dados.

Deploy Publicação de uma aplicação num servidor para que fique acessível aos usuários.

Firewall Sistema de segurança de rede que filtra o tráfego e pode bloquear o acesso a determinados serviços.

Framework Conjunto de bibliotecas e convenções que oferece uma estrutura pronta para o desenvolvimento de aplicações.

Front end Parte do sistema exibida no navegador, com a qual o usuário interage diretamente.

Hash Valor gerado por uma função que transforma um dado, como uma senha, de forma que não seja possível recuperar o original.

Lance automático Lance dado pelo próprio sistema em nome do participante de um leilão, até o valor máximo que ele informou.

Layout Disposição dos elementos numa tela ou página.

Login Identificação do usuário no sistema, normalmente por nome de usuário e senha.

Marketplace Plataforma de comércio eletrônico em que diversos vendedores oferecem seus produtos a compradores.

Migration No Django, arquivo gerado a partir dos modelos que descreve as alterações a aplicar na estrutura do banco de dados.

ngrok Serviço que cria um túnel entre a internet e um servidor local, tornando-o acessível por um endereço público temporário.

Oferta de segunda chance Oferta feita pelo vendedor a outro participante de um leilão que não foi vendido ou não foi pago.

Pé de galinha Notação de diagramas entidade-relacionamento que representa a cardinalidade com símbolos nas extremidades das linhas, como um tridente para "muitos"; em inglês, *crow's foot*.

Preço de reserva Valor mínimo, definido pelo vendedor, abaixo do qual o item leiloado não é vendido.

Sandbox Ambiente de testes isolado, em que as operações não têm efeito real; no Mercado Pago, permite simular pagamentos.

Script Arquivo de código interpretado; no texto, os arquivos JavaScript executados no navegador.

Stakeholder Pessoa ou grupo interessado no projeto ou afetado por ele.

Template Arquivo HTML com marcações especiais que o Django preenche com os dados de cada página antes de enviá-la ao navegador.

Token Código usado para autenticar o acesso de uma aplicação a um serviço externo.

Volta Valor em dinheiro que complementa uma troca quando os itens trocados têm valores diferentes.

Webhook Notificação enviada automaticamente por um serviço externo a um endereço do sistema quando ocorre um evento, como a confirmação de um pagamento.

Wireframe Esboço simplificado de uma tela, que mostra a disposição dos elementos sem o acabamento visual.